تحلیل جامع زمان‌بندی طولانی‌ترین کار ابتدا (Longest Job First – LJF): اصول، ویژگی‌های عملکردی و ارزیابی مقایسه‌ای با زمان‌بندی زودترین مهلت ابتدا (Earliest Deadline First – EDF) و نرخ یکنوا (Rate Monotonic – RM)

یک دست‌نوشته علمی با کیفیت انتشار

صادق عضدی

تاریخ ارسال دست‌نوشته: *[تاریخ]*

واژگان کلیدی: طولانی‌ترین کار ابتدا (LJF)، زودترین مهلت ابتدا (EDF)، نرخ یکنوا (RM)، زمان‌بندی CPU، سیستم‌های بلادرنگ، زمان‌بندی چندپردازنده‌ای، گرسنگی (Starvation)، بن‌بست (Deadlock)

طبقه‌بندی پژوهشی: سیستم‌های عامل، زمان‌بندی بلادرنگ، ارزیابی عملکرد

۰. چکیده

این دست‌نوشته یک بررسی جامع و از نظر فنی دقیق از الگوریتم زمان‌بندی طولانی‌ترین کار ابتدا (LJF) ارائه می‌دهد و ارزیابی مقایسه‌ای آن را با دو سیاست بنیادین زمان‌بندی بلادرنگ، یعنی زودترین مهلت ابتدا (EDF) و نرخ یکنوا (RM)، فراهم می‌کند. مطالعه در دو بخش اصلی سازمان‌دهی شده است. بخش اول تحلیلی مستقل و عمیق از LJF ارائه می‌کند که شامل مدل صوری، گونه‌های الگوریتمی (LJF غیرپیش‌گیرانه و طولانی‌ترین زمان باقیمانده ابتدا (LRTF) پیش‌گیرانه)، فرمول‌بندی ریاضی، پیچیدگی محاسباتی، ویژگی‌های گرسنگی، سازگاری‌های چندپردازنده‌ای و مثال‌های عددی کارشده با نمودارهای گانت و شبه‌کد است. بخش دوم یک مقایسه کنترل‌شده برقرار می‌کند و روشن می‌سازد که LJF به طول تخمینی اجرای کار اولویت می‌دهد، در حالی که EDF به‌صورت پویا وظیفه‌ای با زودترین مهلت مطلق را انتخاب می‌کند و RM اولویت‌های ثابت را بر اساس دوره‌های وظیفه تخصیص می‌دهد. دست‌نوشته شامل آزمون‌های صوری زمان‌بندی‌پذیری، جداول مقایسه‌ای، مثال‌های مجموعه‌وظیفه تکرارپذیر و بحثی انتقادی درباره چندپردازش، گرسنگی، بن‌بست و وارونگی اولویت است. تمام مثال‌های کمی به‌صورت تحلیلی استخراج شده و به‌طور مستقل قابل راستی‌آزمایی هستند. این سند به‌عنوان یک دست‌نوشته علمی HTML۵ خودکفا با نمودارهای SVG جاسازی‌شده، قالب‌بندی آماده چاپ A۴ و منابع به سبک IEEE طراحی شده است.

واژگان کلیدی: طولانی‌ترین کار ابتدا (LJF)، زودترین مهلت ابتدا (EDF)، نرخ یکنوا (RM)، زمان‌بندی CPU، سیستم‌های بلادرنگ، زمان‌بندی چندپردازنده‌ای، گرسنگی، بن‌بست، وارونگی اولویت، تحلیل زمان‌بندی‌پذیری.

۰. فهرست مطالب

بخش اول — مطالعه جامع الگوریتم طولانی‌ترین کار ابتدا (LJF)

۱. مقدمه و انگیزه پژوهش

۲. زمینه زمان‌بندی CPU و مدیریت فرآیند

۳. تعریف و اصول بنیادین LJF

۴. زمینه تاریخی و مفهومی

۵. مدل صوری زمان‌بندی و مفروضات

۶. فرمول‌بندی ریاضی و قواعد تصمیم

۷. گونه‌های الگوریتم LJF

۸. رویه اجرای گام‌به‌گام

۹. ویژگی‌ها و خواص اصلی

۱۰. مزایا و معایب

۱۱. پیچیدگی محاسباتی و پیاده‌سازی

۱۲. معیارهای عملکرد

۱۳. انصاف، گرسنگی و عدم تعادل بار

۱۴. چندپردازشی و زمان‌بندی چندپردازنده‌ای

۱۵. سیستم‌های چند هسته‌ای، تعادل بار و وابستگی پردازنده

۱۶. رقابت منابع و همگام‌سازی

۱۷. بن‌بست: تعریف و ارتباط

۱۸. کاربردهای عملی و محدودیت‌ها

۱۹. مثال عددی کارشده

۲۰. نمودار گانت و تفسیر آن

۲۱. جدول زمان‌بندی فرآیند

۲۲. شبه‌کد

۲۳. فلوچارت

۲۴. معیارهای عملکرد و تحلیل ریاضی

۲۵. بحث انتقادی و پیامدهای پژوهشی

بخش دوم — تحلیل مقایسه‌ای LJF، EDF و RM

۲۶. زودترین مهلت ابتدا (EDF)

۲۷. نرخ یکنوا (RM)

۲۸. تفاوت مفهومی بنیادین

۲۹. جداول مقایسه‌ای

۳۰. مقایسه عددی کنترل‌شده: مثال مجموعه‌وظیفه EDF و RM

۳۱. نمودارهای گانت و اشکال مقایسه‌ای

۳۲. چندپردازشی، گرسنگی، بن‌بست و وارونگی اولویت

۳۳. روش‌شناسی پژوهش و تکرارپذیری

۳۴. نتایج و بحث

۳۵. محدودیت‌ها و تهدیدهای اعتبار

۳۶. توصیه‌های عملی

۳۷. جهت‌گیری‌های پژوهشی آینده

۳۸. نتیجه‌گیری

۳۹. منابع

پیوست الف: محاسبات تکمیلی

پیوست ب: شبه‌کد تکمیلی و یادداشت‌های تکرارپذیری

۱. مقدمه و انگیزه پژوهش

زمان‌بندی CPU یکی از ارکان طراحی سیستم‌های عامل است و به‌طور مستقیم بر پاسخ‌گویی سیستم، گذردهی (Throughput)، انصاف و پیش‌بینی‌پذیری تأثیر می‌گذارد. در میان سیاست‌های متعدد زمان‌بندی که در ادبیات موضوع مطالعه شده‌اند، الگوریتم طولانی‌ترین کار ابتدا (LJF) جایگاه متمایزی دارد: این الگوریتم از میان صف آماده، فرآیندی را انتخاب می‌کند که بزرگ‌ترین زمان انفجار (Burst) تخمینی CPU را دارد. با وجود سادگی مفهومی، LJF اغلب بدفهمیده می‌شود، غالباً با گونه پیش‌گیرانه آن — طولانی‌ترین زمان باقیمانده ابتدا (LRTF) — اشتباه گرفته می‌شود و به‌ندرت با همان دقتی که الگوریتم‌های زمان‌بندی بلادرنگ مانند زودترین مهلت ابتدا (EDF) و نرخ یکنوا (RM) تحلیل می‌شود، بررسی شده است. این دست‌نوشته این شکاف را با ارائه یک بررسی جامع و مستقل از LJF و سپس تحلیل مقایسه‌ای منظم با EDF و RM پر می‌کند.

انگیزه این مطالعه سه‌وجهی است. نخست، LJF به‌عنوان نقطه مقابل آموزنده سیاست رایج‌تر کوتاه‌ترین کار ابتدا (SJF) عمل می‌کند و تأثیر عمیق معیار انتخاب بر زمان انتظار، گرسنگی و انصاف را نشان می‌دهد. دوم، LJF گهگاه در زمینه‌های پردازش دسته‌ای (Batch) و غیرتعاملی پیشنهاد می‌شود که در آن‌ها به حداقل رساندن تعداد کارهای طولانی در سیستم مطلوب است، اما محدودیت‌های آن به‌ندرت با دقت ریاضی بیان می‌شود. سوم، جامعه زمان‌بندی بلادرنگ آزمون‌های پیچیده زمان‌بندی‌پذیری برای EDF و RM توسعه داده است و مقایسه این نتایج با رفتار عمومی LJF مستلزم تفکیک دقیق مفروضات است. این دست‌نوشته بنابراین LJF را در بخش اول به‌صورت مستقل و پیش از هر مقایسه‌ای بررسی می‌کند و سپس در بخش دوم چارچوب مقایسه‌ای کنترل‌شده‌ای برقرار می‌سازد.

پرسش‌های پژوهشی راهنمای این کار عبارت‌اند از: (۱) ویژگی‌های صوری، مشخصات محاسباتی و محدودیت‌های عملی LJF غیرپیش‌گیرانه و LRTF پیش‌گیرانه چیست؟ (۲) LJF از نظر هدف بهینه‌سازی، اطلاعات ورودی موردنیاز و مدل بار کاری چه تفاوت بنیادینی با EDF و RM دارد؟ (۳) تحت چه شرایطی می‌توان مقایسه‌های کمی را بدون درهم‌آمیختن مفروضات ناسازگار انجام داد؟ (۴) پیامدهای LJF برای سیستم‌های چندپردازنده‌ای، گرسنگی، بن‌بست و وارونگی اولویت چیست؟ دست‌نوشته به این پرسش‌ها از طریق تعاریف صوری، مثال‌های کارشده، شبه‌کد، فلوچارت و جداول مقایسه‌ای پاسخ می‌دهد.

۲. زمینه زمان‌بندی CPU و مدیریت فرآیند

در یک سیستم عامل چندبرنامه‌ای، زمان‌بند CPU فرآیندی را از صف آماده انتخاب کرده و پردازنده‌ای به آن تخصیص می‌دهد. تصمیم زمان‌بندی توسط رویدادهایی مانند ایجاد فرآیند، خاتمه، مسدود شدن برای ورودی/خروجی یا انقضای یک برش زمانی فعال می‌شود. سیاست‌های زمان‌بندی معمولاً در چند محور طبقه‌بندی می‌شوند: پیش‌گیرانه در برابر غیرپیش‌گیرانه، اولویت ثابت در برابر پویا، و کارپذیر (Work-conserving) در برابر غیرکارپذیر. انتخاب سیاست بر معیارهای عملکرد بنیادین از جمله بهره‌وری CPU، گذردهی، زمان بازگشت (Turnaround Time)، زمان انتظار و زمان پاسخ تأثیر می‌گذارد.

بررسی‌های کلاسیک کتاب‌های درسی، الگوریتم‌های زمان‌بندی را به سیاست‌های دسته‌ای (اول‌آمده اول‌خدمت‌گرفته (FCFS)، کوتاه‌ترین کار ابتدا (SJF)، طولانی‌ترین کار ابتدا (LJF))، سیاست‌های تعاملی (نوبت گردشی (Round Robin)، زمان‌بندی اولویت‌دار) و سیاست‌های بلادرنگ (زودترین مهلت ابتدا (EDF)، نرخ یکنوا (RM)) گروه‌بندی می‌کنند. LJF به‌طور سنتی در کنار SJF به‌عنوان نقطه مقابل ارائه می‌شود: در حالی که SJF میانگین زمان انتظار را برای مجموعه‌ای از کارها که هم‌زمان می‌رسند کمینه می‌کند، LJF تمایل دارد آن را بیشینه کند. این رفتار حدی، LJF را به یک ابزار آموزشی ارزشمند و یک آزمون تنش (Stress Test) برای شبیه‌سازهای زمان‌بندی تبدیل می‌کند، هرچند به‌ندرت در سیستم‌های تعاملی عمومی به کار می‌رود.

مدیریت فرآیند شامل نمایش فرآیندها (بلوک‌های کنترل فرآیند (PCB))، انتقال‌های حالت (جدید، آماده، در حال اجرا، انتظار، خاتمه‌یافته)، تعویض زمینه (Context Switch) و نگهداری صف‌های آماده است. LJF با بررسی زمان انفجار تخمینی CPU هر فرآیند مقیم در صف آماده عمل می‌کند. در سیستم‌هایی که زمان‌های انفجار از پیش معلوم نیستند، LJF به سازوکارهای پیش‌بینی مانند میانگین‌گیری نمایی نیاز دارد که خطای تخمین را وارد کرده و تحلیل را پیچیده می‌کند. این دست‌نوشته برای قابلیت تحلیل، زمان‌های انفجار را معلوم یا به‌طور کامل تخمین‌زده‌شده فرض می‌کند و به‌صراحت مشخص می‌کند که این فرض در عمل کجا شکسته می‌شود.

۳. تعریف و اصول بنیادین LJF

تعریف (طولانی‌ترین کار ابتدا غیرپیش‌گیرانه). با داشتن مجموعه‌ای از فرآیندهای آماده که هر یک زمان انفجار کل CPU تخمینی \(B_i \) دارند، LJF غیرپیش‌گیرانه فرآیندی را با بیشترین \(B_i \) انتخاب کرده و آن را تا اتمام اجرا می‌کند. تساوی‌ها با یک قاعده قطعی، معمولاً ترتیب اولین‌آمده اول‌خدمت‌گرفته (FCFS) یا شناسه فرآیند، شکسته می‌شوند. پس از شروع اجرای یک فرآیند، تا زمان اتمام یا مسدود شدن داوطلبانه، پیش‌گیری نمی‌شود.

تعریف (طولانی‌ترین زمان باقیمانده ابتدا). طولانی‌ترین زمان باقیمانده ابتدا (LRTF) گونه پیش‌گیرانه LJF است. در هر لحظه زمان‌بندی، زمان‌بند فرآیند آماده‌ای را با بیشترین زمان اجرای باقیمانده انتخاب می‌کند. اگر فرآیند تازه‌واردی زمان باقیمانده‌اش از فرآیند در حال اجرا بیشتر باشد، پیش‌گیری رخ می‌دهد. LRTF گاهی همتای کوتاه‌ترین زمان باقیمانده ابتدا (SRTF) نامیده می‌شود، هرچند این تقارن کامل نیست زیرا LRTF به‌ندرت برای هر هدف استانداردی بهینه است.

تفکیک LJF از LRTF ضروری است. LJF غیرپیش‌گیرانه تا اتمام به یک کار متعهد می‌ماند، در حالی که LRTF ممکن است یک کار طولانی در حال اجرا را به نفع کار طولانی‌تری که تازه رسیده است پیش‌گیری کند. این دو الگوریتم زمان‌بندی‌های متفاوتی تولید می‌کنند، پروفایل‌های گرسنگی متفاوتی دارند و به ساختارهای داده پیاده‌سازی متفاوتی نیاز دارند. در سراسر این دست‌نوشته، اصطلاح «LJF» به‌طور خاص به گونه غیرپیش‌گیرانه اشاره دارد مگر آنکه صراحتاً «LJF پیش‌گیرانه (LRTF)» ذکر شود.

LJF اساساً به کارهای طولانی بر کارهای کوتاه اولویت می‌دهد. این نقطه مقابل SJF است که برای مجموعه‌ای از کارهای هم‌زمان‌رسیده، میانگین زمان انتظار را کمینه می‌کند. در مقابل، LJF تمایل دارد میانگین زمان انتظار را متورم کند و می‌تواند زمانی که کارهای طولانی همچنان می‌رسند، گرسنگی شدید کارهای کوتاه را سبب شود. با این حال، LJF این ویژگی را دارد که تعداد کارهای طولانی باقیمانده در سیستم را سریع کاهش می‌دهد، که ممکن است در برخی محیط‌های دسته‌ای که کارهای طولانی پرمصرف هستند و اتمام زودهنگام آن‌ها منابع را آزاد می‌کند، مطلوب باشد.

۴. زمینه تاریخی و مفهومی

خاستگاه‌های LJF و SJF را می‌توان به سیستم‌های پردازش دسته‌ای اولیه دهه‌های ۱۹۵۰ و ۱۹۶۰ ردیابی کرد که در آن‌ها زمان بازگشت کار دغدغه اصلی بود. کار بنیادین کانوی، مکسول و میلر در نظریه زمان‌بندی، این سیاست‌ها را صوری‌سازی کرد و ثابت کرد که SJF برای مجموعه‌ای ثابت از کارها میانگین زمان جریان (Flow Time) را کمینه می‌کند. LJF عمدتاً به‌عنوان یک همتای نظری و به‌عنوان یک معیار بدترین حالت (Worst-case Benchmark) پدیدار شد. در ادبیات سیستم‌های بلادرنگ، تمرکز به‌سوی سیاست‌های مبتنی بر مهلت معطوف شد و در مقاله بنیادین لیو و لیلند در سال ۱۹۷۳ به اوج رسید که زمان‌بندی نرخ یکنوا (RM) را معرفی کرد و بهینگی آن را در میان سیاست‌های اولویت ثابت برای وظایف دوره‌ای با مهلت‌های مساوی دوره‌ها اثبات کرد. EDF نیز در همان دوران تحلیل شد و بعدها نشان داده شد که برای زمان‌بندی پیش‌گیرانه تک‌پردازنده‌ای وظایف دوره‌ای مستقل، زمانی که بهره‌وری کل از ۱ تجاوز نکند، بهینه است.

LJF یک الگوریتم زمان‌بندی بلادرنگ مبتنی بر مهلت نیست. این الگوریتم مهلت‌ها، دوره‌ها یا هیچ محدودیت زمانی دیگری جز طول انفجار تخمینی را در نظر نمی‌گیرد. نقش تاریخی آن بنابراین عمدتاً آموزشی و تحلیلی است: پیامدهای یک قاعده انتخاب حریصانه خاص را نشان می‌دهد و نقطه مقابلی برای SJF و سیاست‌های مبتنی بر مهلت فراهم می‌کند. در سیستم‌های عامل مدرن، LJF در زمان‌بندهای عمومی مانند زمان‌بند کاملاً منصفانه لینوکس (CFS) یا زمان‌بند اولویت‌دار پیش‌گیرانه ویندوز استفاده نمی‌شود. با این حال، مفهوم اولویت‌دهی به کارهای طولانی در حوزه‌های خاص مانند ابررایانه‌های دسته‌ای و برخی زمان‌بندهای کار موازی ظاهر می‌شود، جایی که هدف ممکن است پاک‌سازی زودهنگام کارهای بزرگ برای بهبود کل زمان اتمام (Makespan) باشد.

۵. مدل صوری زمان‌بندی و مفروضات

یک سیستم تک‌پردازنده‌ای با مجموعه‌ای از \(n \) فرآیند مستقل \(P = \{P_۱, P_۲, \ldots, P_n\} \) را در نظر می‌گیریم. هر فرآیند \(P_i \) با موارد زیر مشخص می‌شود:

- زمان ورود \(a_i \): زمانی که \(P_i \) برای اجرا آماده می‌شود.
- زمان انفجار (\(B_i \) CPU): کل زمان اجرای موردنیاز \(P_i \) (معلوم یا به‌طور کامل تخمین‌زده‌شده فرض می‌شود).
- زمان اجرای باقیمانده \(R_i(t) \): مقدار اجرای باقیمانده \(P_i \) در زمان \(t \). در ابتدا \(R_i(a_i) = B_i \).
- زمان شروع \(s_i \): زمانی که \(P_i \) نخستین بار CPU دریافت می‌کند.
- زمان اتمام \(c_i \): زمانی که \(P_i \) اجرای خود را به پایان می‌رساند.
- زمان بازگشت \(T_i = c_i - a_i \).
- زمان انتظار \(W_i = T_i - B_i \).
- زمان پاسخ \(rho_i = s_i - a_i \).

برای وظایف بلادرنگ، علاوه بر این یک مهلت نسبی \(D_i \)، یک دوره \(T_i \) و یک مهلت مطلق \(d_i = a_i + D_i \) برای هر نمونه وظیفه تعریف می‌کنیم. تعداد پردازنده‌ها با \(m \) نشان داده می‌شود؛ در بخش اول، \(m = ۱ \) است مگر آنکه ملاحظات چندپردازنده‌ای صراحتاً بررسی شود.

مفروضات استاندارد زیر در صورت عدم ذکر خلاف آن اعمال می‌شوند: (۱) فرآیندها مستقل هستند و منابعی را به اشتراک نمی‌گذارند؛ (۲) سربار تعویض زمینه ناچیز است؛ (۳) زمان‌های انفجار دقیقاً معلوم هستند؛ (۴) همه فرآیندها CPU-محور هستند و برای ورودی/خروجی مسدود نمی‌شوند؛ (۵) زمان‌بند کارپذیر است. تحت این مفروضات، معادلات استاندارد برقرارند:

$$T_i = c_i - a_i \tag{۱}$$

$$W_i = T_i - B_i \tag{۲}$$

$$rho_i = s_i - a_i \tag{۳}$$

$$\bar{W} = \frac{۱}{n} \sum_{i=۱}^{n} W_i \tag{۴}$$

$$\bar{T} = \frac{۱}{n} \sum_{i=۱}^{n} T_i \tag{۵}$$

این معادلات فرآیندهای تک‌انفجاری بدون مسدودسازی را فرض می‌کنند. هنگامی که ورودی/خروجی یا مسدودسازی وجود دارد، زمان انتظار باید زمان صرف‌شده در حالت انتظار را در نظر بگیرد و فرمول ساده \(W_i = T_i - B_i \) دیگر تنها انتظار CPU را نشان نمی‌دهد.

۶. فرمول‌بندی ریاضی و قواعد تصمیم

فرض کنید \(\mathcal{R}(t) \) مجموعه فرآیندهای آماده در زمان \(t \) را نشان دهد. برای LJF غیرپیش‌گیرانه، زمان‌بند انتخاب می‌کند:

$$P^{*} = \arg\max_{P_i \in \mathcal{R}(t)} B_i \quad (۶)$$

با یک قاعده قطعی شکستن تساوی مانند \(\arg\min_i a_i \) (زودترین ورود) یا \(\arg\min_i i \) (کمترین شاخص). برای LRTF، معیار انتخاب از زمان باقیمانده استفاده می‌کند:

$$P^{*} = \arg\max_{P_i \in \mathcal{R}(t)} R_i(t) \quad (۷)$$

زمان‌بند وضعیت سیستم را به‌صورت زیر به‌روزرسانی می‌کند. اگر \(\mathcal{R}(t) \) ناتهی باشد، \(P^{*} \) برای مدتی \(\Delta = \min(\Delta_{\text{رویداد بعدی}}, R_{P^{*}}(t)) \) اجرا می‌شود. در حالت غیرپیش‌گیرانه، \(\Delta = R_{P^{*}}(t) \) است مگر آنکه فرآیند مسدود شود. زمان اتمام \(P^{*} \) برابر است با \(c_{P^{*}} = s_{P^{*}} + B_{P^{*}} \). در هر رویداد ورود یا اتمام، مجموعه آماده بازمحاسبه شده و قاعده تصمیم دوباره اعمال می‌شود.

برای مجموعه‌ای از \(n \) کار که همه در زمان ۰ می‌رسند، زمان‌بندی LJF کارها را به ترتیب نزولی زمان انفجار مرتب می‌کند: \(B_{(۱)} \ge B_{(۲)} \ge \dots \ge B_{(n)} \). زمان اتمام کار \(k \)-ام در این ترتیب برابر است با:

$$c_{(k)} = \sum_{j=۱}^{k} B_{(j)} \quad (۸)$$

زمان انتظار کار \(k \)-ام برابر است با \(W_{(k)} = c_{(k)} - B_{(k)} = \sum_{j=۱}^{k-۱} B_{(j)} \). در نتیجه، میانگین زمان انتظار برابر است با:

$$\bar{W} = \frac{۱}{n} \sum_{k=۱}^{n} \sum_{j=۱}^{k-۱} B_{(j)} = \frac{۱}{n} \sum_{j=۱}^{n-۱} (n-j)\, B_{(j)} \quad (۹)$$

این عبارت نشان می‌دهد که LJF کارهای طولانی‌تر را در مجموع زمان انتظار برای کارهایی که دیرتر در ترتیب قرار می‌گیرند، سنگین‌تر وزن می‌دهد. از آنجا که \(B_{(۱)} \) بزرگ‌ترین انفجار است، ضریب \((n-۱) \) در بزرگ‌ترین انفجار ضرب می‌شود، که تمایل دارد میانگین زمان انتظار را در مقایسه با SJF متورم کند، جایی که در عبارت متناظر، کوچک‌ترین انفجارها بزرگ‌ترین ضرایب را دریافت می‌کنند.

۷. گونه‌های الگوریتم LJF

۷.۱ طولانی‌ترین کار ابتدا غیرپیش‌گیرانه

این الگوریتم متعارف LJF است. یک فرآیند، پس از انتخاب، تا اتمام (یا تا مسدود شدن برای ورودی/خروجی، در صورت مجاز بودن مسدودسازی) اجرا می‌شود. LJF غیرپیش‌گیرانه پیاده‌سازی ساده‌ای دارد: صف آماده می‌تواند به‌صورت یک صف اولویت‌دار (Priority Queue) با کلید زمان انفجار نگهداری شود. سادگی آن به بهای پاسخ‌گویی ضعیف برای کارهای کوتاه و گرسنگی احتمالی تمام می‌شود. در سیستمی با جریان پیوسته کارهای طولانی، یک کار کوتاه که در زمان \(t \) می‌رسد، ممکن است به‌طور نامحدود منتظر بماند اگر کارهای طولانی‌تری پیش از انتخاب آن برسند.

۷.۲ طولانی‌ترین زمان باقیمانده ابتدا (LRTF)

LRTF فرآیند در حال اجرا را پیش‌گیری می‌کند اگر فرآیند تازه‌واردی زمان اجرای باقیمانده بزرگ‌تری داشته باشد. این گونه نیازمند آن است که زمان‌بند در هر رویداد ورود، زمان‌های باقیمانده را مقایسه کرده و تخمین‌های دقیق زمان باقیمانده را نگهداری کند. LRTF به ورود کارهای طولانی پاسخ‌گوتر است اما می‌تواند پیش‌گیری مکرر کارهای نسبتاً طولانی را سبب شود و سربار تعویض زمینه را افزایش دهد. مهم است که LRTF با LJF اشتباه گرفته نشود: در LRTF، کاری که اجرا را آغاز کرده ممکن است معلق شود، در حالی که در LJF تا اتمام اجرا می‌شود.

۷.۳ تخمین‌های ایستا و پویا از طول کار

در عمل، زمان‌های انفجار از پیش معلوم نیستند. LJF می‌تواند با استفاده از تخمین‌های ایستا (مثلاً راهنمایی‌های ارائه‌شده توسط برنامه‌نویس یا میانگین‌های کلاس کار) یا تخمین‌های پویا (مثلاً میانگین‌گیری نمایی از انفجارهای گذشته) سازگار شود. تخمین‌های ایستا ساده‌اند اما ممکن است نادرست باشند؛ تخمین‌های پویا با گذشت زمان سازگار می‌شوند اما خطای تخمین را وارد کرده و به وضعیت اضافی نیاز دارند. انتخاب روش تخمین بر رفتار الگوریتم تأثیر می‌گذارد: تخمین‌های خوش‌بینانه از کارهای طولانی ممکن است گرسنگی را تشدید کند، در حالی که تخمین‌های بدبینانه ممکن است زمان‌بند را به اولویت‌دهی بیش‌ازحد به کارهایی که سریع تمام می‌شوند سوق دهد.

۸. رویه اجرای گام‌به‌گام

رویه زیر LJF غیرپیش‌گیرانه را برای مجموعه‌ای از فرآیندها با زمان‌های ورود و انفجار معلوم توصیف می‌کند. یک پردازنده واحد و عدم مسدودسازی برای ورودی/خروجی فرض شده است.

۱. **مقداردهی اولیه.** زمان جاری \(t = ۰ \) را تنظیم کنید. همه فرآیندها را به‌عنوان واردنشده و ناتمام علامت بزنید. یک صف آماده خالی \(Q \) ایجاد کنید.

۲. **پیشبرد ورودها.** همه فرآیندهایی با زمان ورود \(a_i \le t \) را از مجموعه واردنشده به \(Q \) منتقل کنید.

۳. **بررسی صف آماده.** اگر \(Q \) خالی است و فرآیندهای واردنشده وجود دارند، \(t \) را به زمان ورود بعدی تنظیم کرده و به گام ۲ بروید. اگر \(Q \) خالی است و فرآیندی باقی نمانده، خاتمه دهید.

۴. **انتخاب فرآیند.** از \(Q \)، فرآیندی با بیشترین زمان انفجار \(B_i \) را انتخاب کنید. تساوی‌ها را با زودترین زمان ورود و سپس کمترین شناسه فرآیند بشکنید.

۵. **اجرا.** فرآیند انتخاب‌شده را از \(Q \) حذف کنید. زمان شروع آن را \(s_i = t \) قرار دهید. آن را برای \(B_i \) واحد زمانی اجرا کنید. \(t = t + B_i \) را تنظیم کنید. زمان اتمام آن را \(c_i = t \) قرار دهید.

۶. **ثبت معیارها.** زمان بازگشت \(T_i = c_i - a_i \)، زمان انتظار \(W_i = T_i - B_i \) و زمان پاسخ \(rho_i = s_i - a_i \) را محاسبه کنید.

۷. **تکرار.** به گام ۲ بروید.

هنگامی که ورودها به‌صورت پویا رخ می‌دهند، صف آماده در هر رویداد ورود به‌روزرسانی می‌شود. از آنجا که LJF غیرپیش‌گیرانه هرگز فرآیند در حال اجرا را پیش‌گیری نمی‌کند، کار طولانی‌تری که تازه رسیده است باید تا اتمام کار جاری منتظر بماند. این تفاوت کلیدی رفتاری با LRTF است که در صورت بیشتر بودن زمان باقیمانده کار جدید از زمان باقیمانده کار در حال اجرا، پیش‌گیری می‌کرد.

۹. ویژگی‌ها و خواص اصلی

LJF چندین ویژگی قابل توجه دارد. نخست، در شکل متعارف خود **غیرپیش‌گیرانه** است، یعنی تعویض زمینه تنها در زمان اتمام کار (یا مسدودسازی داوطلبانه) رخ می‌دهد. دوم، **حریصانه** است: در هر نقطه تصمیم، کاری را انتخاب می‌کند که از نظر تقاضای CPU «پرهزینه‌تر» به نظر می‌رسد. سوم، برای هیچ هدف زمان‌بندی استانداردی روی یک تک‌پردازنده **بهینه نیست**، مگر به معنای بدیهی که زمانی که همه کارها هم‌زمان می‌رسند، زمان اتمام کوتاه‌ترین کار را بیشینه می‌کند. چهارم، **مستعد گرسنگی** است: کارهای کوتاه می‌توانند به‌طور نامحدود به تعویق بیفتند اگر کارهای طولانی‌تر همچنان برسند. پنجم، برای کارهای کوتاه **رفتار بدترین حالت قابل پیش‌بینی** دارد، که آن را به‌عنوان یک معیار خصمانه (Adversarial Benchmark) مفید می‌کند.

برای مجموعه‌ای از \(n \) کار که همه در زمان ۰ می‌رسند، LJF کارها را به ترتیب نزولی زمان انفجار مرتب می‌کند. میانگین زمان انتظار با معادله (۹) داده می‌شود. این را می‌توان با SJF مقایسه کرد که کارها را به ترتیب صعودی زمان انفجار مرتب می‌کند و به دست می‌آورد:

$$\bar{W}_{\text{SJF}} = \frac{1}{n} \sum_{j=1}^{n-1} (n-j) B_{(j)}^{\text{asc}} \quad (۱۰)$$

از آنجا که \(B_{(j)}^{\text{asc}} \) \(j \)-امین کوچک‌ترین انفجار است، بزرگ‌ترین ضرایب در کوچک‌ترین انفجارها ضرب می‌شوند، که دلیل کمینه‌سازی میانگین زمان انتظار توسط SJF است. LJF عکس این کار را انجام می‌دهد، که دلیل بیشینه‌سازی میانگین زمان انتظار توسط آن در میان همه سیاست‌های غیرپیش‌گیرانه‌ای است که کارها را با یک قاعده اولویت ثابت برای ورودهای هم‌زمان مرتب می‌کنند.

۱۰. مزایا و معایب

۱۰.۱ مزایا

- **سادگی**. LJF غیرپیش‌گیرانه با استفاده از یک صف اولویت‌دار بسیار ساده پیاده‌سازی می‌شود.
- **سربار کم**. تعویض زمینه کمتر نسبت به سیاست‌های پیش‌گیرانه، که در سیستم‌هایی که هزینه تعویض زمینه بالاست می‌تواند مفید باشد.
- **قابل پیش‌بینی برای کارهای طولانی**. کارهای طولانی اولویت می‌گیرند، بنابراین زودتر از SJF یا FIFO تمام می‌شوند، که می‌تواند در محیط‌های دسته‌ای که کارهای طولانی بر مصرف منابع غالب‌اند مفید باشد.
- **ارزش آموزشی**. LJF تضاد روشنی با SJF فراهم می‌کند و پیامدهای انتخاب حریصانه را نشان می‌دهد.

۱۰.۲ معایب

- **گرسنگی**. کارهای کوتاه می‌توانند به‌طور نامحدود به تعویق بیفتند اگر کارهای طولانی همچنان برسند.
- **میانگین زمان انتظار بالا**. برای ورودهای هم‌زمان، LJF میانگین زمان انتظار را در میان ترتیب‌های بر اساس طول انفجار بیشینه می‌کند.
- **پاسخ‌گویی ضعیف**. وظایف تعاملی یا کوتاه زمان پاسخ طولانی را تجربه می‌کنند.
- **بدون آگاهی از مهلت**. LJF هیچ تضمینی برای برآورده کردن مهلت‌ها ارائه نمی‌دهد و بدون اصلاح برای سیستم‌های بلادرنگ نامناسب است.
- **حساسیت به خطای تخمین**. تخمین‌های نادرست زمان انفجار می‌توانند زمان‌بند را به تصمیم‌های ضعیف سوق دهند.

۱۱. پیچیدگی محاسباتی و ملاحظات پیاده‌سازی

برای مجموعه‌ای از \(n \) فرآیند با زمان‌های ورود و انفجار معلوم، LJF غیرپیش‌گیرانه می‌تواند با استفاده از یک پشته بیشینه (Max-heap) با کلید زمان انفجار پیاده‌سازی شود. هر فرآیند یک بار درج و یک بار استخراج می‌شود و پیچیدگی زمانی کل برای خود تصمیم‌های زمان‌بندی \(O(n \log n) \) است. اگر ورودها پویا باشند و زمان‌بند باید رویدادها را به‌صورت برخط مدیریت کند، پیچیدگی هر رویداد برای درج و استخراج \(O(\log n) \) است. پیچیدگی فضایی برای ذخیره رکوردهای فرآیند و پشته \(O(n) \) است.

برای LRTF، زمان‌بند باید علاوه بر این زمان‌های باقیمانده را به‌روزرسانی کرده و در هر ورود احتمالاً پیش‌گیری کند. می‌توان از یک صف اولویت‌دار با کلید زمان باقیمانده استفاده کرد، اما زمان باقیمانده فرآیند در حال اجرا به‌طور پیوسته تغییر می‌کند. در عمل، زمان‌بند می‌تواند فرآیند در حال اجرا را خارج از پشته نگه دارد و در هر ورود زمان باقیمانده آن را با بیشینه پشته مقایسه کند. پیچیدگی هر ورود برای درج \(O(\log n) \) و برای مقایسه \(O(۱) \) است. سربار تعویض زمینه نسبت به LJF غیرپیش‌گیرانه بیشتر است زیرا اگر ورودها انفجاری باشند، پیش‌گیری‌ها می‌توانند مکرراً رخ دهند.

ملاحظات پیاده‌سازی شامل موارد زیر است: (۱) قواعد شکستن تساوی باید قطعی باشند تا زمان‌بندی‌های تکرارپذیر تضمین شوند؛ (۲) تخمین‌های زمان انفجار باید نگهداری و به‌روزرسانی شوند؛ (۳) صف آماده باید در سیستم‌های چندپردازنده‌ای با همگام‌سازی مناسب محافظت شود؛ (۴) زمان‌بند باید کارپذیر باشد، یعنی هرگز CPU را در صورت وجود فرآیند آماده بی‌کار رها نکند.

۱۲. معیارهای عملکرد

معیارهای زیر در سراسر این دست‌نوشته برای توصیف LJF و مقایسه آن با EDF و RM استفاده می‌شوند. تعاریف در بخش ۵ و معادلات (۱) تا (۵) آمده‌اند. برای مجموعه‌ای از \(n \) فرآیند، گزارش می‌کنیم:

- **میانگین زمان انتظار** \(\bar{W} \): میانگین زمانی که فرآیندها در صف آماده سپری می‌کنند.
- **میانگین زمان بازگشت** \(\bar{T} \): میانگین زمان از ورود تا اتمام.
- **میانگین زمان پاسخ** \(\bar{\rho} \): میانگین زمان از ورود تا اولین اجرا.
- **بهره‌وری CPU** \(U \): کسری از زمان که CPU مشغول اجرای فرآیندهاست. برای مجموعه متناهی کار، \(U = \sum B_i / \text{کل زمان اتمام} \). برای وظایف دوره‌ای، \(U = \sum C_i / T_i \).
- **گذردهی**: تعداد فرآیندهای تکمیل‌شده در واحد زمان.
- **کل زمان اتمام (Makespan)**: زمان اتمام آخرین فرآیند.
- **تعداد مهلت‌های ازدست‌رفته**: فقط برای مجموعه‌وظایف بلادرنگ با مهلت‌ها قابل اعمال است.
- **کل تأخیر (Total Tardiness)**: مجموع تفاوت‌های مثبت بین زمان‌های اتمام و مهلت‌ها.

همه معیارها برای همه الگوریتم‌ها معنادار نیستند. برای مثال، مهلت‌های ازدست‌رفته برای LJF وقتی که به فرآیندهای عمومی بدون مهلت اعمال می‌شود، تعریف‌نشده است، مگر آنکه نگاشت از فرآیندها به وظایف بلادرنگ به‌صراحت تعریف شود. این دست‌نوشته به‌دقت معیارهایی را که به‌طور جهانی قابل اعمال هستند از آن‌هایی که مدل‌خاص هستند تفکیک می‌کند.

۱۳. انصاف، گرسنگی و عدم تعادل بار

گرسنگی زمانی رخ می‌دهد که یک فرآیند به‌طور نامحدود از پیشرفت بازداشته شود زیرا زمان‌بند به‌طور مستمر فرآیندهای دیگر را انتخاب می‌کند. LJF ذاتاً مستعد گرسنگی کارهای کوتاه است: اگر کارهای طولانی همچنان برسند، یک کار کوتاه ممکن است هرگز فرآیند با بیشترین انفجار در صف آماده نباشد. این یک نقص نیست بلکه پیامد مستقیم قاعده انتخاب است. گرسنگی باید از انتظار عادی (که در مجموعه متناهی کار محدود است)، وارونگی اولویت (که در آن یک وظیفه با اولویت بالا برای یک منبع در اختیار وظیفه‌ای با اولویت پایین منتظر می‌ماند) و بن‌بست (که در آن فرآیندها به‌طور نامحدود برای منابع در اختیار یکدیگر منتظر می‌مانند) متمایز شود.

راهبردهای کاهش گرسنگی در LJF عبارت‌اند از:

- **پیرسازی (Aging):** افزایش تدریجی اولویت فرآیندهای منتظر، به‌گونه‌ای که در نهایت با کارهای طولانی رقابتی شوند. پیرسازی LJF را به یک سیاست ترکیبی تبدیل می‌کند که دیگر LJF خالص نیست.
- **انتظار محدود:** اعمال حداکثر زمان انتظار که پس از آن یک فرآیند ارتقا می‌یابد. این نیز سیاست را تغییر می‌دهد.
- **سیاست‌های ترکیبی:** ترکیب LJF با SJF یا نوبت گردشی در یک صف چندسطحی. برای مثال، کارهای کوتاه می‌توانند توسط یک صف SJF جداگانه سرویس داده شوند در حالی که کارهای طولانی از LJF استفاده می‌کنند.
- **کنترل پذیرش:** محدود کردن نرخ ورود کارهای طولانی یا رزرو ظرفیت برای کارهای کوتاه.

هر راهبرد کاهش، رفتار الگوریتم را تغییر می‌دهد و باید به‌عنوان یک سیاست متمایز تحلیل شود. LJF خالص، طبق تعریف، هیچ سازوکار پیرسازی یا انتظار محدودی ندارد. عدم تعادل بار نیز یک نگرانی دیگر است: اگر بار کاری عمدتاً از کارهای طولانی تشکیل شده باشد، LJF ممکن است از نظر پاک‌سازی کارهای طولانی به‌طور معقولی عمل کند، اما کارهای کوتاه آسیب خواهند دید. در مقابل، اگر بار کاری تحت سلطه کارهای کوتاه باشد، LJF آن‌ها را به نفع چند کار طولانی به تعویق می‌اندازد و به میانگین زمان پاسخ ضعیف منجر می‌شود.

۱۴. چندپردازشی و زمان‌بندی چندپردازنده‌ای

گسترش LJF به سیستم‌های چندپردازنده‌ای مستلزم مشخص‌سازی دقیق معماری زمان‌بندی است. در سیستمی با \(m \) پردازنده یکسان، یک صف آماده مشترک و زمان‌بندی متمرکز، زمان‌بند می‌تواند \(m \) کار طولانی‌تر آماده را در هر نقطه تصمیم به \(m \) پردازنده تخصیص دهد. این گاهی «طولانی‌ترین کار ابتدا با \(m \) پردازنده» یا «طولانی‌ترین زمان پردازش» (LPT) در ادبیات زمان‌بندی ماشین‌های موازی نامیده می‌شود. LPT یک هیوریستیک شناخته‌شده برای کمینه‌سازی کل زمان اتمام روی ماشین‌های موازی یکسان است؛ این هیوریستیک کارها را به ترتیب نزولی زمان پردازش مرتب کرده و هر کار را به اولین ماشین در دسترس تخصیص می‌دهد. LPT برای کمینه‌سازی کل زمان اتمام نسبت تقریب \(۴/۳ - ۱/(۳m) \) دارد که بهتر از زمان‌بندی فهرستی (List Scheduling) است اما در حالت کلی هنوز بهینه نیست.

تفکیک زمان‌بندی فرآیند CPU از زمان‌بندی ماشین‌های موازی بسیار مهم است. در زمان‌بندی CPU، فرآیندها ممکن است مستقل باشند یا ارتباط برقرار کنند؛ در زمان‌بندی ماشین‌های موازی، کارها معمولاً مستقل هستند و هدف اغلب کل زمان اتمام است. LJF در زمینه زمان‌بندی CPU چندپردازنده‌ای ممکن است با صف‌های هر پردازنده، یک صف جهانی با وابستگی پردازنده، یا یک توزیع‌کننده متمرکز پیاده‌سازی شود. هر طراحی پیامدهای متفاوتی برای تعادل بار، گرسنگی و سربار دارد. تخصیص کارهای طولانی به پردازنده‌های جداگانه می‌تواند کل زمان اتمام را کاهش دهد، اما ممکن است اگر کارهای طولانی مدت‌های متفاوتی داشته باشند، عدم تعادل بار را نیز افزایش دهد.

برای پردازنده‌های ناهمگن (سرعت‌ها یا قابلیت‌های متفاوت)، LJF باید برای در نظر گرفتن وابستگی پردازنده و سرعت نسبی سازگار شود. انتخاب طولانی‌ترین کار ابتدا روی یک پلتفرم چندپردازنده‌ای معادل تضمین زمان‌بندی بهینه تحت هر هدفی نیست. در واقع، زمان‌بندی چندپردازنده‌ای برای بسیاری از اهداف NP-سخت است و LJF/LPT هیوریستیک‌هایی هستند نه الگوریتم‌های بهینه.

۱۵. سیستم‌های چند هسته‌ای، تعادل بار و وابستگی پردازنده

پردازنده‌های چند هسته‌ای مدرن ملاحظات اضافی را وارد می‌کنند. وابستگی حافظه نهان (Cache Affinity)، اثرات NUMA و چندرشته‌ای هم‌زمان (Simultaneous Multithreading) می‌توانند پردازنده‌ها را از دید زمان‌بند نایکسان کنند. LJF باید تصمیم بگیرد که آیا فرآیندها را بین هسته‌ها برای تعادل بار مهاجرت دهد یا آن‌ها را روی یک هسته نگه دارد تا محلی‌بودن حافظه نهان را بهره‌برداری کند. مهاجرت یک کار طولانی ممکن است وضعیت حافظه نهان آن را مختل کرده و زمان اجرا را افزایش دهد، بنابراین وابستگی پردازنده ممکن است حتی اگر به عدم تعادل بار موقت منجر شود ترجیح داده شود. یک رویکرد رایج استفاده از یک صف جهانی برای تخصیص اولیه و صف‌های هر هسته برای زمان‌بندی بعدی، با تعادل بار دوره‌ای که محدودیت‌های وابستگی را رعایت می‌کند، است.

گرسنگی در LJF چند هسته‌ای می‌تواند به‌صورت هر هسته‌ای یا جهانی رخ دهد. اگر هر هسته صف آماده خود را نگه دارد، یک کار کوتاه روی هسته‌ای با کارهای طولانی ممکن است گرسنه بماند حتی اگر هسته‌های دیگر بی‌کار باشند. دزدی کار جهانی (Global Work-stealing) یا بازتوازن دوره‌ای صف‌ها می‌تواند این را کاهش دهد، اما سربار وارد می‌کند. زمان‌بند باید هزینه تعویض زمینه و گرم شدن حافظه نهان را نیز هنگام تصمیم‌گیری درباره پیش‌گیری یا مهاجرت در نظر بگیرد. این ملاحظات عملی به این معناست که LJF در سیستم‌های چند هسته‌ای به‌ندرت خالص است؛ معمولاً در یک چارچوب زمان‌بندی بزرگ‌تر که شامل سازوکارهای انصاف و تعادل بار است جاسازی می‌شود.

۱۶. رقابت منابع و همگام‌سازی

LJF، مانند هر سیاست زمان‌بندی CPU، در محیطی عمل می‌کند که فرآیندها ممکن است برای منابع مشترک مانند قفل‌ها، سمافورها و دستگاه‌های ورودی/خروجی رقابت کنند. رقابت منابع می‌تواند فرآیندها را مسدود کند و آن‌ها را از صف آماده حذف کند. هنگامی که یک فرآیند مسدود می‌شود، زمان‌بند باید فرآیند آماده دیگری را طبق قاعده LJF انتخاب کند. این تعامل بین زمان‌بندی و مسدودسازی می‌تواند تحلیل را پیچیده کند: یک کار طولانی که پیش‌گیری یا مسدود شده است ممکن است زمان باقیمانده‌اش تغییر کند، و یک کار کوتاه که قفلی را به دست می‌آورد ممکن است یک کار طولانی را به تأخیر بیندازد و به اثرات شبیه وارونگی اولویت منجر شود.

پروتکل‌های همگام‌سازی مانند ارث‌بری اولویت (Priority Inheritance) و سقف اولویت (Priority Ceiling) معمولاً برای سیستم‌های اولویت ثابت طراحی شده‌اند. LJF مفهوم طبیعی اولویت فراتر از طول انفجار ندارد، بنابراین سازگار کردن این پروتکل‌ها مستلزم تعریف یک اولویت کمکی است. بدون چنین سازوکارهایی، رقابت منابع می‌تواند به مسدودسازی نامحدود منجر شود که ممکن است شبیه گرسنگی باشد اما ناشی از وابستگی‌های منابع است نه فقط سیاست زمان‌بندی. این دست‌نوشته تأکید می‌کند که LJF خودش نه رقابت منابع را ایجاد می‌کند و نه از آن جلوگیری می‌کند؛ ترکیب زمان‌بندی و همگام‌سازی است که رفتار سیستم را تعیین می‌کند.

۱۷. بن‌بست: تعریف و ارتباط

بن‌بست حالتی است که در آن مجموعه‌ای از فرآیندها به‌طور دائم مسدود شده‌اند زیرا هر یک برای منبعی در اختیار دیگری در همان مجموعه منتظر است. چهار شرط لازم کلاسیک برای بن‌بست عبارت‌اند از:

۱. **انحصار متقابل (Mutual Exclusion)**: حداقل یک منبع در حالت غیرقابل اشتراک نگهداری می‌شود.

۲. **نگهداری و انتظار (Hold and Wait)**: یک فرآیند حداقل یک منبع را نگه می‌دارد در حالی که برای منابع اضافی منتظر است.

۳. **عدم پیش‌گیری (No Preemption)**: منابع نمی‌توانند به‌زور از یک فرآیند گرفته شوند؛ آن‌ها باید داوطلبانه آزاد شوند.

۴. **انتظار دایره‌ای (Circular Wait)**: زنجیره بسته‌ای از فرآیندها وجود دارد که هر فرآیند منبعی را نگه می‌دارد که فرآیند بعدی در زنجیره به آن نیاز دارد.

EDF، LJF و RM سیاست‌های زمان‌بندی CPU هستند، نه الگوریتم‌های پیشگیری یا اجتناب از بن‌بست. آن‌ها تعیین می‌کنند که کدام فرآیند آماده روی CPU اجرا شود؛ آن‌ها کنترل نمی‌کنند که منابع چگونه تخصیص می‌یابند یا آیا انتظار دایره‌ای می‌تواند رخ دهد. انتخاب طولانی‌ترین کار ابتدا نه ذاتاً بن‌بست ایجاد می‌کند و نه از آن جلوگیری می‌کند. بن‌بست به گراف تخصیص منابع و پروتکل همگام‌سازی بستگی دارد، نه به قاعده زمان‌بندی CPU. یک سیستم با استفاده از LJF می‌تواند بن‌بست شود اگر چهار شرط برقرار باشند، و یک سیستم با استفاده از EDF می‌تواند تحت همان شرایط تخصیص منابع بن‌بست شود. بنابراین نسبت دادن بن‌بست به یک سیاست زمان‌بندی CPU خاص نادرست است. بن‌بست باید از طریق ترتیب‌دهی منابع، الگوریتم‌های اجتناب (مانند الگوریتم بانکدار)، تشخیص و بازیابی، یا پروتکل‌های همگام‌سازی بدون بن‌بست مورد توجه قرار گیرد.

گرسنگی، در مقابل، یک پدیده سطح زمان‌بندی است که می‌تواند توسط سیاست زمان‌بندی CPU تحت تأثیر قرار گیرد. LJF می‌تواند گرسنگی کارهای کوتاه را سبب شود، در حالی که EDF می‌تواند در شرایط اضافه‌بار مهلت‌ها را از دست بدهد، و RM می‌تواند در صورت مدیریت نادرست منابع مشترک وارونگی اولویت را سبب شود. این‌ها از بن‌بست متمایز هستند.

۱۸. کاربردهای عملی و محدودیت‌ها

LJF به‌ندرت در سیستم‌های عامل عمومی استفاده می‌شود زیرا تمایل دارد وظایف کوتاه و تعاملی را جریمه کند. با این حال، کاربردهای خاصی در پردازش دسته‌ای دارد، جایی که کارهای طولانی پرمصرف هستند و اتمام زودهنگام آن‌ها منابع سیستم را آزاد می‌کند. در زمان‌بندهای کار موازی، LPT (طولانی‌ترین زمان پردازش) یک هیوریستیک شناخته‌شده برای کمینه‌سازی کل زمان اتمام است. LJF همچنین در زمینه‌های آموزشی به‌عنوان نقطه مقابل SJF و در شبیه‌سازهای زمان‌بندی به‌عنوان آزمون تنش ظاهر می‌شود. محدودیت‌های آن قابل توجه است: گرسنگی، پاسخ‌گویی ضعیف، حساسیت به خطای تخمین و عدم آگاهی از مهلت. در سیستم‌های بلادرنگ، LJF انتخاب مناسبی نیست مگر آنکه با اطلاعات مهلت و کنترل پذیرش دقیق تقویت شود، که در آن نقطه عملاً به سیاست دیگری تبدیل می‌شود.

۱۹. مثال عددی کارشده

پنج فرآیند \(P_۱, P_۲, P_۳, P_۴, P_۵ \) را در نظر بگیرید که همه در زمان \(t = ۰ \) می‌رسند. زمان‌های انفجار عبارت‌اند از: \(B_۱ = ۱۰ \)، \(B_۲ = ۴ \)، \(B_۳ = ۷ \)، \(B_۴ = ۲ \)، \(B_۵ = ۶ \). ما LJF غیرپیش‌گیرانه را با شکستن تساوی بر اساس کمترین شناسه فرآیند اعمال می‌کنیم (هرچند در اینجا تساوی رخ نمی‌دهد). ترتیب اجرا بر اساس کاهش زمان انفجار است: \(P_۱ \) (۱۰)، \(P_۳ \) (۷)، \(P_۵ \) (۶)، \(P_۲ \) (۴)، \(P_۴ \) (۲).

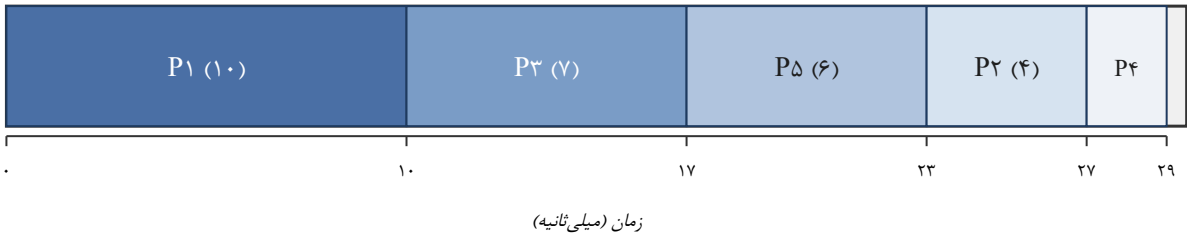
فرآیند	ورود \(a_i \)	انفجار \(B_i \)	شروع \(s_i \)	اتمام \(c_i \)	بازگشت \(T_i \)	انتظار \(W_i \)	پاسخ \(rho_i \)
P۱	۰	۱۰	۰	۱۰	۱۰	۰	۰
P۳	۰	۷	۱۰	۱۷	۱۷	۱۰	۱۰
P۵	۰	۶	۱۷	۲۳	۲۳	۱۷	۱۷
P۲	۰	۴	۲۳	۲۷	۲۷	۲۳	۲۳
P۴	۰	۲	۲۷	۲۹	۲۹	۲۷	۲۷

جدول ۱: مثال زمان‌بندی LJF با همه فرآیندها که در زمان ۰ می‌رسند.

میانگین زمان انتظار: \(۷۷/۵ = ۱۵.۴ \). \(\bar{W} = (۰ + ۱۰ + ۱۷ + ۲۳ + ۲۷)/۵ \). میانگین زمان بازگشت: \(\bar{T} = (۱۰ + ۱۷ + ۲۳ + ۲۷)/۵ = ۱۰۶/۵ = ۲۱.۲ \). میانگین زمان پاسخ: \(\bar{\rho} = (۰ + ۱۰ + ۱۷ + ۲۳ + ۲۷)/۵ = ۱۵.۴ \) (برابر با زمان انتظار زیرا همه ورودها در ۰ هستند و زمان‌بندی غیرپیش‌گیرانه است). کل زمان اتمام = ۲۹. بهره‌وری CPU = ۲۹/۲۹ = ۱.۰.

برای مقایسه، SJF روی همان مجموعه به ترتیب \(P_۱(۱۰), P_۳(۷), P_۵(۶), P_۲(۴), P_۴(۲) \) مرتب می‌کند و میانگین زمان انتظار \((۰ + ۲ + ۶ + ۱۲ + ۱۹)/۵ = ۳۹/۵ = ۷.۸ \) را به دست می‌دهد. LJF تقریباً میانگین زمان انتظار را دو برابر می‌کند که عملکرد ضعیف آن را در این معیار نشان می‌دهد.

۲۰. نمودار گانت و تفسیر آن



شکل ۱: نمودار گانت برای LJF غیرپیش‌گیرانه با پنج فرآیند که در زمان ۰ می‌رسند. ترتیب اجرا: P۱، P۳، P۵، P۲، P۴

نمودار گانت ترتیب اجرا و زمان‌بندی جدول ۱ را تأیید می‌کند. کارهای طولانی P۱ و P۳ ابتدا اجرا می‌شوند و کارهای کوتاه P۲ و P۴ را تا انتها به تعویق می‌اندازند. این نشانه مشخص LJF است: کارهای کوتاه زمان انتظار طولانی را تجربه می‌کنند. هیچ بازه بی‌کاری وجود ندارد زیرا همه کارها در زمان ۰ می‌رسند و CPU کارپذیر است.

۲۱. جدول زمان‌بندی فرآیند

جدول ۱ (ارائه‌شده در بخش ۱۹) جدول زمان‌بندی کامل برای مثال کارشده را فراهم می‌کند. هر ورودی از نظر ریاضی با معادلات (۱) تا (۳) سازگار است. برای مثال، برای P۳: زمان بازگشت \(۱۷ − ۰ = ۱۷ \)، زمان انتظار \(۱۷ − ۷ = ۱۰ \)، زمان پاسخ \(۱۰ − ۰ = ۱۰ \). جدول شامل تمام ستون‌های موردنیاز است: شناسه فرآیند، زمان ورود، انفجار CPU، زمان شروع، زمان اتمام، زمان بازگشت، زمان انتظار و زمان پاسخ.

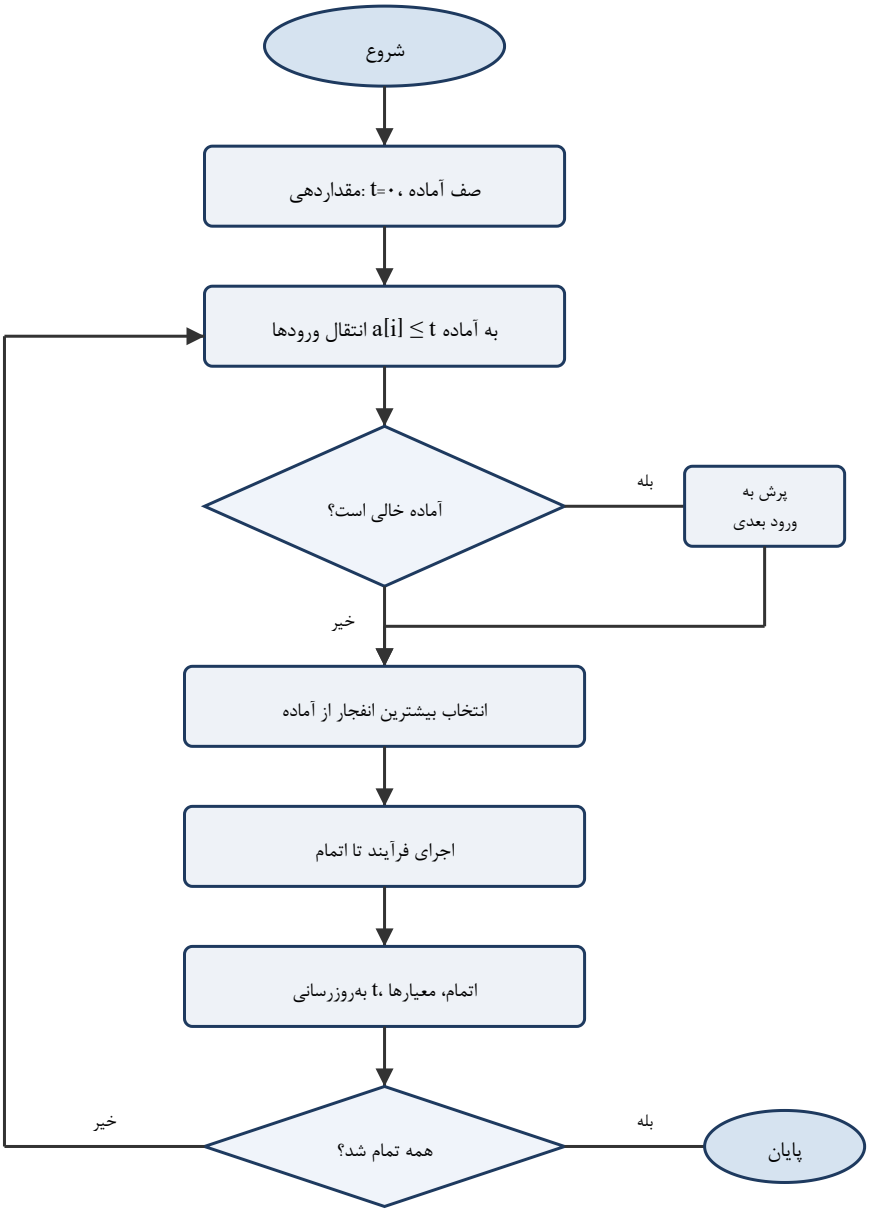
۲۲. شبه‌کد

```
ALGORITHM NonPreemptiveLJF(processes P[۱..n], arrival a[۱..n], burst B[۱..n])
// Assumes single processor, no I/O, known burst times.
// Returns schedule and metrics.

۱. t ← ۰
۲. ready ← empty max-heap keyed by burst time
۳. unfinished ← {۱, ۲, ..., n}
۴. while unfinished is not empty or ready is not empty do
۵.     for each i in unfinished with a[i] ≤ t do
۶.         insert i into ready (key = B[i])
۷.         remove i from unfinished
۸.     if ready is empty then
۹.         t ← min{ a[i] : i in unfinished }   // jump to next arrival
۱۰.        continue
۱۱.    i ← extract-max(ready)                  // tie: earliest arrival, then ID
۱۲.    s[i] ← t
۱۳.    t ← t + B[i]
۱۴.    c[i] ← t
۱۵.    T[i] ← c[i] - a[i]
۱۶.    W[i] ← T[i] - B[i]
۱۷.    ρ[i] ← s[i] - a[i]
۱۸.    record i as completed
۱۹. end while
۲۰. return (s, c, T, W, ρ)
```

پیچیدگی زمانی: \(O(n \log n) \) به دلیل درج‌ها و استخراج‌های پشته. **پیچیدگی فضایی:** \(O(n) \) برای پشته و رکوردهای فرآیند. الگوریتم فرض می‌کند که زمان‌های انفجار معلوم هستند و ورودها از پیش معلوم‌اند یا می‌توانند به‌صورت برخط تشخیص داده شوند. اگر ورودها پویا باشند، زمان‌بند در هر رویداد اجرا می‌شود و منطق یکسانی اعمال می‌گردد. برای LRTF، کلید زمان باقیمانده است و فرآیند در حال اجرا ممکن است پیش‌گیری شود؛ شبه‌کد شامل یک مقایسه در هر ورود و یک گام پیش‌گیری احتمالی خواهد بود.

۲۳. فلوچارت



شکل ۲: فلوچارت زمان‌بندی LJF غیرپیش‌گیرانه. تصمیم‌ها به‌صورت لوزی و گام‌های پردازش به‌صورت مستطیل نشان داده شده‌اند.

۲۴. معیارهای عملکرد و تحلیل ریاضی

برای مثال کارشده در بخش ۱۹، معیارهای LJF عبارت‌اند از: میانگین زمان انتظار \(\bar{W} = ۱۵.۴ \)، میانگین زمان بازگشت \(\bar{T} = ۲۱.۲ \)، میانگین زمان پاسخ \(\bar{\rho} = ۱۵.۴ \)، بهره‌وری (CPU) \(U = ۱.۰ \)، گذردهی \(۵/۲۹ \approx ۰.۱۷۲ \) فرآیند در واحد زمان، و کل زمان اتمام \(۲۹ \). همان مجموعه کار تحت SJF به \(\bar{W} = ۷.۸ \)، \(\bar{T} = ۱۳.۶ \) و بهره‌وری و کل زمان اتمام یکسان می‌رسد. این به‌صورت کمی برتری SJF بر LJF را در معیارهای زمان انتظار برای ورودهای هم‌زمان نشان می‌دهد. فرمول معادله (۹) می‌تواند برای راستی‌آزمایی میانگین زمان انتظار استفاده شود: \(\bar{W} = (۴ \cdot ۱۰ + ۳ \cdot ۷ + ۲ \cdot ۶ + ۱ \cdot ۴ + ۰ \cdot ۰)/۵ = (۴۰ + ۲۱ + ۱۲ + ۴ + ۰)/۵ = ۷۷/۵ = ۱۵.۴ \).

از نظر گرسنگی، کوتاه‌ترین کار P۴ (انفجار ۲) ۲۷ واحد زمانی منتظر می‌ماند که بیش از انفجار آن با ضریب ۱۳.۵ است. اگر کارهای طولانی همچنان می‌رسیدند، P۴ می‌توانست به‌طور نامحدود منتظر بماند. این ریسک گرسنگی را نشان می‌دهد. گذردهی LJF برای این مجموعه متناهی با SJF یکسان است زیرا کل کار یکسان است؛ تفاوت کاملاً در توزیع زمان‌های انتظار نهفته است. برای معیارهای بلادرنگ، LJF بدون مهلت‌ها قابل اعمال نیست، بنابراین مهلت‌های ازدست‌رفته و تأخیر در این مثال تعریف‌نشده‌اند.

۲۵. بحث انتقادی و پیامدهای پژوهشی

تحلیل در بخش اول تأیید می‌کند که LJF یک سیاست ساده و غیرپیش‌گیرانه است که به کارهای طولانی اولویت می‌دهد. ضعف اصلی آن گرسنگی کارهای کوتاه و میانگین زمان انتظار ضعیف است. قدرت اصلی آن سادگی و سربار کم است. از منظر پژوهشی، LJF به‌عنوان یک خط پایه مهم عمل می‌کند: هر سیاست زمان‌بندی پیشنهادی که ادعای بهبود انصاف یا پاسخ‌گویی را دارد باید با LJF مقایسه شود تا نشان دهد از رفتار آسیب‌شناختی LJF اجتناب می‌کند. LJF همچنین اهمیت تفکیک سیاست زمان‌بندی از مدیریت منابع را برجسته می‌کند: بن‌بست و وارونگی اولویت با انتخاب LJF، EDF یا RM حل نمی‌شوند؛ آن‌ها به پروتکل‌های اختصاصی نیاز دارند.

یک پیامد مهم برای سیستم‌های بلادرنگ این است که LJF یک الگوریتم زمان‌بندی مبتنی بر مهلت نیست. این الگوریتم مهلت‌ها را در نظر نمی‌گیرد و بنابراین نمی‌تواند تضمین‌های زمان‌بندی‌پذیری ارائه دهد. هر تلاشی برای استفاده از LJF در زمینه بلادرنگ مستلزم تقویت آن با اطلاعات مهلت و کنترل پذیرش است که در آن نقطه دیگر LJF خالص نیست. این دست‌نوشته بنابراین توصیه می‌کند که LJF به‌عنوان یک سیاست زمان‌بندی عمومی دسته‌ای و نه به‌عنوان یک سیاست بلادرنگ تلقی شود. تحلیل مقایسه‌ای در بخش دوم تفاوت‌های بنیادین بین LJF، EDF و RM را بیشتر روشن خواهد کرد.

۲۶. زودترین مهلت ابتدا (EDF)

زودترین مهلت ابتدا (EDF) یک الگوریتم زمان‌بندی با اولویت پویا برای سیستم‌های بلادرنگ است. در هر لحظه زمان‌بندی، EDF وظیفه آماده‌ای را با زودترین مهلت مطلق \(d_i = a_i + D_i \) انتخاب می‌کند، که در آن \(a_i \) زمان ورود (آزادسازی) و \(D_i \) مهلت نسبی است. EDF در شکل استاندارد خود پیش‌گیرانه است: اگر وظیفه تازه‌آزادشده‌ای مهلت زودتری از وظیفه در حال اجرا داشته باشد، آن را پیش‌گیری می‌کند. EDF می‌تواند به‌صورت غیرپیش‌گیرانه نیز اعمال شود، اما نتیجه بهینگی کلاسیک، پیش‌گیری را فرض می‌کند.

EDF برای مدل‌های وظیفه دوره‌ای، پراکنده (Sporadic) و غیردوره‌ای (Aperiodic) تعریف شده است. در مدل دوره‌ای، هر وظیفه \(\tau_i \) با زمان اجرای بدترین حالت \(C_i \)، دوره \(T_i \) و مهلت نسبی \(D_i \) مشخص می‌شود. بهره‌وری وظیفه \(U_i = C_i / T_i \) و بهره‌وری کل \(U = \sum_{i=1}^{n} U_i \) است. برای حالت خاصی که مهلت‌ها مساوی دوره‌ها هستند (\(D_i = T_i \)) و وظایف مستقل و پیش‌گیرپذیرند، EDF روی یک تک‌پردازنده بهینه است: اگر یک زمان‌بندی امکان‌پذیر وجود داشته باشد، EDF آن را پیدا خواهد کرد. شرط امکان‌پذیری کلاسیک \(U \le 1 \) است. این نتیجه مستلزم آن است که: (۱) سربار زمان‌بندی ناچیز باشد، (۲) زمان‌های اجرا معلوم باشند، (۳) وظایف مستقل باشند، (۴) زمان‌بندی پیش‌گیرانه باشد، و (۵) مهلت‌ها مساوی دوره‌ها باشند. این نتیجه را نمی‌توان به‌طور indiscriminate به وظایف با مهلت محدود (\(D_i < T_i \))، سیستم‌های غیرپیش‌گیرانه، سیستم‌های اشتراک منابع یا سیستم‌های چندپردازنده‌ای اعمال کرد.

مزایای EDF شامل بهره‌وری بالای CPU، بهینگی روی تک‌پردازنده‌ها تحت مفروضات بیان‌شده، و سازگاری پویا با مهلت‌های متغیر است. معایب شامل سربار زمان اجرای بالاتر به دلیل به‌روزرسانی‌های پویای اولویت، رفتار ضعیف در اضافه‌بار (در اضافه‌بار، EDF می‌تواند آبشاری از مهلت‌های ازدست‌رفته را سبب شود که گاهی «اثر دومینو» نامیده می‌شود) و دشواری در پیش‌بینی اینکه کدام وظایف در اضافه‌بار مهلت را از دست می‌دهند. EDF به‌طور گسترده در سیستم‌های بلادرنگ توکار، رباتیک و کاربردهای چندرسانه‌ای استفاده می‌شود.

۲۷. نرخ یکنوا (RM)

نرخ یکنوا (RM) یک الگوریتم زمان‌بندی با اولویت ثابت برای وظایف دوره‌ای بلادرنگ است. به هر وظیفه یک اولویت ایستا به‌طور معکوس متناسب با دوره آن تخصیص می‌یابد: هرچه دوره کوتاه‌تر باشد، اولویت بالاتر است. RM پیش‌گیرانه است: یک وظیفه با اولویت بالاتر وظیفه با اولویت پایین‌تر را پیش‌گیری می‌کند. مدل وظیفه وظایف دوره‌ای با زمان‌های اجرای بدترین حالت معلوم \(C_i \)، دوره‌ها \(T_i \) و مهلت‌های نسبی \(D_i \) (معمولاً \(D_i = T_i \)) را فرض می‌کند. بهره‌وری کل \(U = \sum C_i / T_i \) است.

کران بهره‌وری کلاسیک لیو-لیلند برای \(n \) وظیفه دوره‌ای مستقل با مهلت‌های مساوی دوره‌ها روی یک تک‌پردازنده عبارت است از:

$$U \le n(2^{1/n} - 1) \quad (11)$$

این یک آزمون کافی زمان‌بندی‌پذیری است: اگر \(U \) کران را برآورده کند، RM زمان‌بندی‌پذیری را تضمین می‌کند. این آزمون لازم نیست؛ برخی مجموعه‌وظایف با بهره‌وری بالاتر از کران ممکن است هنوز زمان‌بندی‌پذیر باشند و تحلیل زمان پاسخ می‌تواند در آن موارد زمان‌بندی‌پذیری را اثبات کند. با \(n \to \infty \)، کران به \(\ln 2 \approx ۰.۶۹۳ \) نزدیک می‌شود. کران برای \(n \) کوچک محافظه‌کارانه است: برای \(n = ۱ \) برابر ۱.۰؛ برای \(n = ۲ \) برابر \(۲(\sqrt{۲} - ۱) \approx ۰.۸۲۸ \)؛ برای \(n = ۳ \) تقریباً ۰.۷۸۰ است.

مزایای RM شامل سادگی، پیش‌بینی‌پذیری و در دسترس بودن آزمون‌های کارآمد زمان‌بندی‌پذیری است. معایب شامل کران بهره‌وری پایین‌تر در مقایسه با EDF، وارونگی اولویت هنگام استفاده از منابع مشترک (که با پروتکل‌های ارث‌بری اولویت یا سقف اولویت کاهش می‌یابد) و نابهینگی برای مجموعه‌وظایف با مهلت‌های متفاوت از دوره‌هاست. RM به‌طور گسترده در اویونیک، سیستم‌های خودرویی و کنترل صنعتی استفاده می‌شود.

۲۸. تفاوت مفهومی بنیادین

EDF، LJF و RM ویژگی‌های متفاوتی را بهینه یا اولویت‌بندی می‌کنند. LJF بر اساس طول تخمینی اجرای کار انتخاب می‌کند: این یک سیاست زمان‌بندی CPU عمومی است که مهلت‌ها یا دوره‌ها را در نظر نمی‌گیرد. EDF بر اساس زودترین مهلت مطلق انتخاب می‌کند: این یک سیاست بلادرنگ با اولویت پویا است که مستقیماً برآورده‌سازی مهلت را هدف قرار می‌دهد. RM اولویت‌های ثابت را بر اساس دوره‌های وظیفه تخصیص می‌دهد: این یک سیاست بلادرنگ با اولویت ایستا است که به وظایف با دوره کوتاه‌تر اولویت می‌دهد، که در میان سیاست‌های اولویت ثابت برای وظایف دوره‌ای با مهلت‌های مساوی دوره‌ها بهینه است. این الگوریتم‌ها قابل تعویض نیستند. مقایسه آن‌ها به‌گونه‌ای که گویی تحت مفروضات یکسانی عمل می‌کنند گمراه‌کننده خواهد بود. LJF عمدتاً با زمان‌بندی عمومی CPU یا دسته‌ای مرتبط است، در حالی که EDF و RM عمدتاً در زمان‌بندی بلادرنگ مطالعه می‌شوند. LJF هدفش پاک‌سازی کارهای طولانی است، EDF هدفش برآورده کردن مهلت‌هاست، و RM هدفش ارائه رفتار قابل پیش‌بینی با اولویت ثابت و یک آزمون ساده زمان‌بندی‌پذیری است.

۲۹. جداول مقایسه‌ای

ویژگی	LJF	EDF	RM
معیار زمان‌بندی	بزرگ‌ترین انفجار تخمینی CPU	زودترین مهلت مطلق	کوتاه‌ترین دوره (بالاترین اولویت)
نوع اولویت	بر اساس طول انفجار	پویا (مبتنی بر مهلت)	ایستا (مبتنی بر دوره)
اولویت ایستا یا پویا	ایستا برای هر کار (اگر انفجار معلوم باشد)	پویا	ایستا
رفتار پیش‌گیرانه یا غیرپیش‌گیرانه	غیرپیش‌گیرانه (LJF)؛ گونه پیش‌گیرانه LRTF	پیش‌گیرانه (استاندارد)	پیش‌گیرانه
اطلاعات ورودی موردنیاز	تخمین‌های انفجار CPU	زمان‌های اجرا، مهلت‌ها، زمان‌های آزادسازی	زمان‌های اجرا، دوره‌ها، مهلت‌ها
مدل بار کاری معمول	کارهای دسته‌ای، فرآیندهای مستقل	وظایف بلادرنگ دوره‌ای/پراکنده	وظایف بلادرنگ دوره‌ای
آگاهی از مهلت	خیر	بله	بله (از طریق دوره‌ها)
ریسک گرسنگی	بالا برای کارهای کوتاه	پایین در بار عادی؛ در اضافه‌بار، مهلت‌های ازدست‌رفته	پایین برای وظایف با اولویت بالا؛ وارونگی اولویت ممکن است
سربار محاسباتی	کم	متوسط (صف اولویت پویا)	کم (اولویت‌های ایستا)
کاربردهای معمول	پردازش دسته‌ای، آموزش، معیارها	بلادرنگ توکار، چندرسانه‌ای، رباتیک	اویونیک، خودرویی، کنترل صنعتی

جدول ۲: ویژگی‌های اصلی الگوریتم‌ها.

جنبه	LJF	EDF	RM
انصاف	ضعیف برای کارهای کوتاه	منصفانه از نظر مهلت	مبتنی بر اولویت؛ منصفانه برای وظایف با اولویت بالا
پاسخ‌گویی	ضعیف برای کارهای کوتاه/تعاملی	خوب برای وظایف فوری	خوب برای وظایف با دوره کوتاه
تضمین مهلت	هیچ	بهینه روی تک‌پردازنده تحت مفروضات	کران زمان‌بندی‌پذیری؛ تحلیل زمان پاسخ
تحلیل زمان‌بندی‌پذیری	قابل اعمال نیست (بدون مهلت)	بهره‌وری ≤ 1 (کافی و لازم تحت مفروضات)	کران بهره‌وری (کافی)؛ تحلیل زمان پاسخ (دقیق)
پیچیدگی پیاده‌سازی	کم	متوسط	کم
رفتار اضافه‌بار	گرسنگی شدید کارهای کوتاه	اثر دومینو؛ ازدست‌رفتگان غیرقابل پیش‌بینی	قابل پیش‌بینی: وظایف با اولویت پایین‌تر ابتدا مهلت را از دست می‌دهند
تناسب چندپردازنده‌ای	هیوریستیک LPT برای کل زمان اتمام	پیچیده؛ بدون بهینگی جهانی	پیچیده؛ بدون بهینگی جهانی

جنبه	LJF	EDF	RM
پیش‌بینی‌پذیری	پایین برای کارهای کوتاه	متوسط	بالا
استقرار معمول	سیستم‌های دسته‌ای، آموزش	بلادرنگ توکار، رباتیک	بلادرنگ ایمنی-بحرانی

جدول ۳: مزایا، محدودیت‌ها و موارد استفاده.

معیار	LJF	SJF (مرجع)	EDF/RM (قابل اعمال نیست)
میانگین زمان انتظار	۱۵.۴	۷.۸	نامعتبر (بدون مهلت)
میانگین زمان بازگشت	۲۱.۲	۱۳.۶	نامعتبر
میانگین زمان پاسخ	۱۵.۴	۷.۸	نامعتبر
بهره‌وری CPU	۱.۰	۱.۰	نامعتبر
گذردهی	۰.۱۷۲	۰.۱۷۲	نامعتبر
کل زمان اتمام	۲۹	۲۹	نامعتبر
مهلت‌های ازدست‌رفته	نامعتبر	نامعتبر	نامعتبر
کل تأخیر	نامعتبر	نامعتبر	نامعتبر

جدول ۴: ارزیابی عملکرد کمی برای مثال LJF (همه ورودها در ۰).

۳۰. مقایسه عددی کنترل‌شده: مثال مجموعه‌وظیفه EDF و RM

اکنون یک مجموعه‌وظیفه بلادرنگ دوره‌ای با سه وظیفه تعریف می‌کنیم. مهلت‌ها مساوی دوره‌ها هستند. همه وظایف در زمان ۰ آزاد می‌شوند و سپس به‌صورت دوره‌ای آزاد می‌شوند.

وظیفه	زمان اجرای بدترین حالت \(C_i \)	دوره \(T_i = D_i \)	بهره‌وری \(C_i/T_i \)	اولویت RM
\(\tau_۱ \)	۱	۴	۰.۲۵	بالاترین (کوتاه‌ترین دوره)
\(\tau_۲ \)	۲	۶	۰.۳۳۳۳	متوسط
\(\tau_۳ \)	۳	۱۲	۰.۲۵	پایین‌ترین (طولانی‌ترین دوره)

جدول ۵: مجموعه‌وظیفه دوره‌ای برای مقایسه EDF و RM.

بهره‌وری کل: \(U = ۰.۲۵ + ۰.۳۳۳۳ + ۰.۲۵ = ۰.۸۳۳۳ \).

زمان‌بندی‌پذیری RM: کران لیو-لیلند برای \(n = ۳ \) برابر \(۳(۲^{۱/۳} - ۱) \approx ۳(۰.۲۵۹۹) = ۰.۷۷۹۷ \) است. از آنجا که \(U = ۰.۸۳۳۳ > ۰.۷۷۹۷ \)، آزمون کران بهره‌وری غیرقطعی است. تحلیل زمان پاسخ را اعمال می‌کنیم. برای RM، اولویت‌ها: \(\tau_۱ > \tau_۲ > \tau_۳ \).

زمان پاسخ \(\tau_۱ \): \(R_۱ = C_۱ = ۱ \).

زمان پاسخ \(\tau_۲ \): \(R_۲ = C_۲ + \lceil R_۲ / T_۱ \rceil C_۱ \). با تکرار: \(R_۲^{(۰)} = ۲ \)؛ \(R_۲^{(۱)} = ۲ + \lceil ۲/۴ \rceil \cdot ۱ = ۳ \)؛ \(R_۲^{(۲)} = ۲ + \lceil ۳/۴ \rceil \cdot ۱ = ۳ \). بنابراین \(R_۲ = ۳ \le T_۲ = ۶ \).

زمان پاسخ \(\tau_۳ \): \(R_۳ = C_۳ + \lceil R_۳/T_۱ \rceil C_۱ + \lceil R_۳/T_۲ \rceil C_۲ \). با تکرار: \(R_۳^{(۰)} = ۳ \)؛ \(R_۳^{(۱)} = ۳ + \lceil ۳/۴ \rceil \cdot ۱ + \lceil ۳/۶ \rceil \cdot ۲ = ۳ + ۱ + ۲ = ۶ \)؛ \(R_۳^{(۲)} = ۳ + \lceil ۶/۴ \rceil \cdot ۱ + \lceil ۶/۶ \rceil \cdot ۲ = ۳ + ۲ + ۲ = ۷ \)؛ \(R_۳^{(۳)} = ۳ + \lceil ۷/۴ \rceil \cdot ۱ + \lceil ۷/۶ \rceil \cdot ۲ = ۳ + ۲ + ۲ = ۹ \)؛ \(R_۳^{(۴)} = ۳ + \lceil ۹/۴ \rceil \cdot ۱ + \lceil ۹/۶ \rceil \cdot ۲ = ۳ + ۳ + ۴ = ۱۰ \)؛ \(R_۳^{(۵)} = ۳ + \lceil ۱۰/۴ \rceil \cdot ۱ + \lceil ۱۰/۶ \rceil \cdot ۲ = ۳ + ۳ + ۴ = ۱۰ \). بنابراین \(R_۳ = ۱۰ \le T_۳ = ۱۲ \). مجموعه‌وظیفه RM-زمان‌بندی‌پذیر است.

زمان‌بندی‌پذیری EDF: از آنجا که \(U = ۰.۸۳۳۳ \le ۱ \)، EDF تحت مفروضات استاندارد (پیش‌گیرانه، مهلت‌های مساوی دوره‌ها، وظایف مستقل) امکان‌پذیر است. یک زمان‌بندی در طول ابردوره (Hyperperiod) \(۱۲ = \text{lcm}(۴,۶,۱۲) \) می‌سازیم.

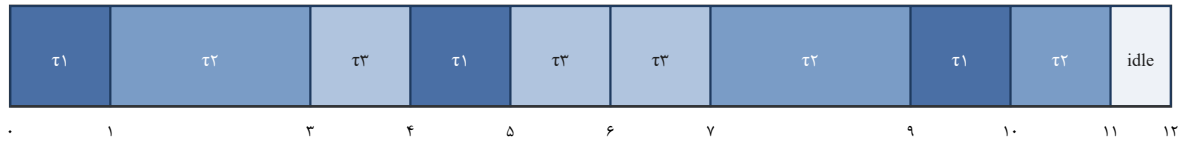
زمان‌بندی EDF در [۰,۱۲]:

- t=۰: آزادسازی \(\tau_۱(۰), \tau_۲(۰), \tau_۳(۰) \). مهلت‌های مطلق: \(d_۱=۴, d_۲=۶, d_۳=۱۲ \). \(\tau_۱ \) EDF را انتخاب می‌کند. اجرا ۰–۱.
- t=۱: \(\tau_۱(۰) \) تمام می‌شود. آماده: \(\tau_۲(۰) \) (d=۶)، \(\tau_۳(۰) \) (d=۱۲). \(\tau_۲ \) را انتخاب می‌کند. اجرا ۱–۳ (۲ واحد).
- t=۳: \(\tau_۲(۰) \) تمام می‌شود. آماده: \(\tau_۳(۰) \) (d=۱۲). \(\tau_۳ \) را انتخاب می‌کند. اجرا ۳–۴ (۱ واحد).

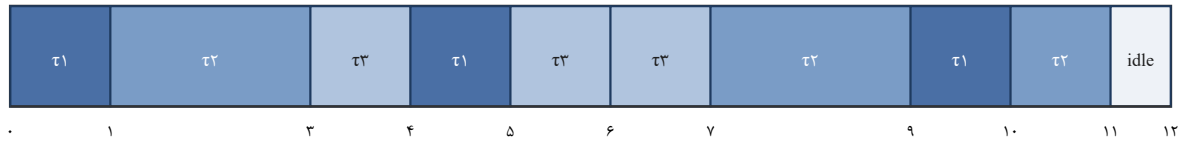
- (\(\tau_{1,1} \)) t=۴: \(\tau_{1,1} \) آزاد می‌شود (d=۸). آماده: \(\tau_{3,0} \) (d=۱۲)، \(\tau_{1,1} \) (d=۸). (\(\tau_1 \)) EDF را انتخاب می‌کند. اجرا ۴–۵.
- (\(\tau_{1,1} \)) t=۵: تمام می‌شود. آماده: \(\tau_{3,0} \) (d=۱۲). \(\tau_{2,1} \) در t=۶ آزاد می‌شود. \(\tau_3 \) را انتخاب می‌کند. اجرا ۵–۶ (۱ واحد).
- (\(\tau_{2,1} \)) t=۶: آزاد می‌شود (d=۱۲). \(\tau_{3,0} \) ۱ واحد باقیمانده دارد. آماده: \(\tau_{3,0} \) (d=۱۲)، \(\tau_{2,1} \) (d=۱۲). تساوی در مهلت؛ فرض می‌کنیم EDF تساوی را با زودترین آزادسازی یا شناسه وظیفه می‌شکند. \(\tau_{3,0} \) آزادسازی زودتری دارد. \(\tau_{3,0} \) را انتخاب می‌کند. اجرا ۶–۷ (۱ واحد).
- (\(\tau_{3,0} \)) t=۷: تمام می‌شود. آماده: \(\tau_{2,1} \) (d=۱۲). \(\tau_{2,1} \) را انتخاب می‌کند. اجرا ۷–۹ (۲ واحد).
- (\(\tau_{1,2} \)) t=۸: آزاد می‌شود (d=۱۲). \(\tau_{2,1} \) را پیش‌گیری می‌کند. \(\tau_{1,2} \) اجرا ۸–۹.
- (\(\tau_{1,2} \)) t=۹: تمام می‌شود. \(\tau_{2,1} \) ۱ واحد باقیمانده دارد. آماده: \(\tau_{2,1} \) (d=۱۲). \(\tau_{3,1} \) در t=۱۲ آزاد می‌شود. \(\tau_{2,1} \) را انتخاب می‌کند. اجرا ۹–۱۰.
- (\(\tau_{2,1} \)) t=۱۰: تمام می‌شود. آماده: هیچ تا t=۱۲. بی‌کاری ۱۰–۱۲.
- (\(\tau_{3,1}, \tau_{2,2}, \tau_{1,3} \)) t=۱۲: آزاد می‌شوند. ابردوره به پایان می‌رسد؛ زمان‌بندی تکرار می‌شود.

همه مهلت‌ها برآورده می‌شوند. هیچ مهلت ازدست‌رفته‌ای وجود ندارد. کل تأخیر = ۰. زمان‌بندی EDF همه مهلت‌ها را برآورده می‌کند، همان‌طور که انتظار می‌رفت چون \(U \le 1 \).

۳۱. نمودارهای گانت و اشکال مقایسه‌ای



شکل ۳: زمان‌بندی EDF برای مجموعه‌وظیفه در طول ابردوره [۰,۱۲). همه مهلت‌ها برآورده شده‌اند؛ بازه بی‌کاری [۱۱,۱۲) پیش از پایان ابردوره.



شکل ۴: زمان‌بندی RM برای مجموعه‌وظیفه در [۰,۱۲). اولویت‌ها: τ۱ بالاترین، τ۲ متوسط، τ۳ پایین‌ترین. همه مهلت‌ها برآورده شده‌اند.



شکل ۵: نمودار مفهومی که معیارهای تصمیم EDF، LJF و RM را نشان می‌دهد.

۳۲. چندپردازشی، گرسنگی، بن‌بست و وارونگی اولویت

۳۲.۱ چندپردازشی

روی سیستم‌های چندپردازنده‌ای، هیچ‌یک از EDF، LJF یا RM تضمین بهینگی جهانی ارائه نمی‌دهد. گونه چندپردازنده‌ای LJF ارتباط نزدیکی با هیوریستیک طولانی‌ترین زمان پردازش (LPT) برای کمینه‌سازی کل زمان اتمام دارد که نسبت تقریب شناخته‌شده‌ای دارد اما بهینه نیست. EDF روی چندپردازنده‌ها پیچیده است: EDF جهانی بهینه نیست و EDF تقسیم‌شده مستلزم تخصیص دقیق وظیفه به پردازنده است. RM روی چندپردازنده‌ها نیز بهینه نیست و کران بهره‌وری برای RM تقسیم‌شده کمتر از تک‌پردازنده است. ادعاهای بهینگی جهانی برای هر یک از این سیاست‌ها روی چندپردازنده‌ها نادرست است.

۳۲.۲ گرسنگی

LJF ریسک بالای گرسنگی برای کارهای کوتاه دارد. EDF و RM معمولاً گرسنگی را به همان معنا سبب نمی‌شوند؛ در عوض، در اضافه‌بار مهلت‌ها را از دست می‌دهند. یک وظیفه که مهلت را از دست می‌دهد لزوماً گرسنه نیست؛ ممکن است زمان CPU دریافت کرده باشد اما برای تکمیل کافی نبوده است. گرسنگی به معنای به تعویق افتادن نامحدود است، که یک پدیده سطح زمان‌بندی است و بیشتر با LJF (و سیستم‌های اولویت ثابت بدون پیرسازی) مرتبط است. مفروضات بار کاری، کنترل پذیرش و سیاست‌های اولویت برای کاهش گرسنگی حیاتی هستند.

۳۲.۳ بن‌بست

همان‌طور که در بخش ۱۷ بحث شد، بن‌بست توسط انتخاب سیاست زمان‌بندی CPU ایجاد یا جلوگیری نمی‌شود. EDF، LJF و RM همگی در صورت برقرار بودن چهار شرط لازم با بن‌بست سازگارند. پیشگیری از بن‌بست مستلزم سازوکارهای ترتیب‌دهی منابع، اجتناب یا تشخیص است که مستقل از زمان‌بند CPU عمل می‌کنند.

۳۲.۴ وارونگی اولویت

وارونگی اولویت زمانی رخ می‌دهد که یک وظیفه با اولویت بالا توسط یک وظیفه با اولویت پایین‌تر که منبع مشترکی را در اختیار دارد مسدود شود و یک وظیفه با اولویت متوسط وظیفه با اولویت پایین را پیش‌گیری کند و زمان مسدودسازی را طولانی‌تر کند. این موضوع به‌ویژه در سیستم‌های اولویت ثابت مانند RM مرتبط است. پروتکل‌های ارث‌بری اولویت و سقف اولویت با افزایش موقت اولویت وظیفه در اختیار منبع، وارونگی اولویت را کاهش می‌دهند. وارونگی اولویت با بن‌بست یا گرسنگی مترادف نیست: این یک تأخیر محدود یا نامحدود ناشی از اشتراک منابع و ترتیب اولویت است. LJF سلسله‌مراتب اولویت طبیعی فراتر از طول انفجار ندارد، بنابراین وارونگی اولویت به معنای کلاسیک کمتر مستقیماً قابل اعمال است، اما رقابت منابع می‌تواند هنوز مسدودسازی نامحدود را در صورت عدم استفاده از پروتکل سبب شود.

۳۳. روش‌شناسی پژوهش و تکرارپذیری

این مطالعه یک مقایسه تحلیلی و توضیحی است، نه یک مطالعه شبیه‌سازی تجربی. روش‌شناسی شامل موارد زیر است: (۱) یک مرور هدفمند ادبیات پژوهش بنیادین زمان‌بندی، شامل لیو و لیلند (۱۹۷۳)، بوتاتزو (۲۰۰۵) و کتاب‌های درسی استاندارد سیستم‌های عامل؛ (۲) تعاریف صوری و استخراج‌های ریاضی قواعد تصمیم EDF، LJF و RM؛ (۳) مثال‌های عددی کارشده که به‌طور مستقل قابل راستی‌آزمایی هستند؛ (۴) ساخت نمودارهای گانت و فلوچارت‌ها با استفاده از SVG؛ و (۵) جداول مقایسه‌ای که نتیجه‌گیری‌ها را بر اساس بار کاری و مفروضات سیستم تعدیل می‌کنند. هیچ نتیجه شبیه‌سازی جعل نشده است. همه مقادیر کمی از مثال‌های کارشده یا از کران‌های کلاسیک زمان‌بندی‌پذیری استخراج شده‌اند. مثال مجموعه‌وظیفه در بخش ۳۰ از یک مدل وظیفه دوره‌ای با مهلت‌های مساوی دوره‌ها استفاده می‌کند و زمان‌بندی‌های EDF و RM به‌صورت دستی در طول ابردوره ساخته شده‌اند. مثال LJF در بخش ۱۹ از پنج فرآیند مستقل با ورودهای هم‌زمان استفاده می‌کند. قواعد شکستن تساوی به‌صراحت بیان شده‌اند. مطالعه تکرارپذیر است: خواننده می‌تواند هر زمان اتمام، زمان انتظار و مقایسه مهلت را با استفاده از معادلات و جداول ارائه‌شده راستی‌آزمایی کند.

۳۴. نتایج و بحث

نتایج این مطالعه را می‌توان به‌صورت زیر خلاصه کرد. نخست، LJF یک سیاست ساده و غیرپیش‌گیرانه است که به کارهای طولانی اولویت می‌دهد و می‌تواند گرسنگی شدید کارهای کوتاه را سبب شود. در مثال کارشده، میانگین زمان انتظار LJF (۱۵.۴) تقریباً دو برابر SJF (۷.۸) برای همان مجموعه کار بود. دوم، LJF یک الگوریتم زمان‌بندی بلادرنگ نیست و هیچ تضمین مهلتی ارائه نمی‌دهد. سوم، EDF و RM سیاست‌های بلادرنگ با آزمون‌های زمان‌بندی‌پذیری به‌خوبی تعریف‌شده هستند: EDF روی تک‌پردازنده‌ها تحت مفروضات استاندارد زمانی که \(U \le ۱ \) باشد بهینه است، و RM یک کران بهره‌وری کافی \(n(۲^{۱/n} - ۱) \) با تحلیل زمان پاسخ برای مجموعه‌وظایفی که از کران فراتر می‌روند ارائه می‌دهد. چهارم، مثال کنترل‌شده مجموعه‌وظیفه نشان داد که هم EDF و هم RM می‌توانند مجموعه‌وظیفه‌ای با \(U = ۰.۸۳۳۳ \) را در طول ابردوره زمان‌بندی کنند و همه مهلت‌ها را برآورده سازند، در حالی که LJF بدون نگاشت از وظایف به کارها نمی‌تواند به‌طور معناداری به آن مجموعه‌وظیفه اعمال شود. پنجم، گسترش‌های چندپردازنده‌ای هر سه سیاست غیربدیهی هستند و بهینگی تک‌پردازنده را به ارث نمی‌برند. ششم، بن‌بست و وارونگی اولویت از انتخاب سیاست زمان‌بندی CPU مستقل هستند و به پروتکل‌های مدیریت منابع جداگانه نیاز دارند.

بحث بر اهمیت عدم مقایسه LJF، EDF و RM به‌گونه‌ای که گویی قابل تعویض هستند تأکید می‌کند. آن‌ها تحت مفروضات متفاوتی عمل می‌کنند و ویژگی‌های متفاوتی را بهینه می‌کنند. LJF به‌عنوان یک خط پایه آموزشی و به‌عنوان یک هیوریستیک زمان‌بندی دسته‌ای مفید است؛ EDF و RM برای سیستم‌های بلادرنگ با محدودیت مهلت مناسب هستند. جداول مقایسه‌ای در بخش ۲۹ این تفاوت‌ها را خلاصه کرده و همه نتیجه‌گیری‌ها را بر اساس بار کاری و مفروضات سیستم مربوطه تعدیل می‌کنند.

۳۵. محدودیت‌ها و تهدیدهای اعتبار

این مطالعه چندین محدودیت دارد. نخست، عمدتاً تحلیلی و توضیحی است؛ هیچ شبیه‌سازی در مقیاس بزرگ یا اندازه‌گیری تجربی انجام نشده است. مقایسه‌های کمی مبتنی بر مثال‌های کوچک کارشده و کران‌های کلاسیک هستند که ممکن است به همه بارهای کاری تعمیم نیابند. دوم، فرض معلوم بودن زمان‌های انفجار برای بسیاری از سیستم‌های عملی غیرواقعی است؛ LJF در عمل به تخمین‌ها تکیه می‌کند که خطا وارد کرده و تحلیل را پیچیده می‌کند. سوم، مثال مجموعه‌وظیفه بلادرنگ از مهلت‌های مساوی دوره‌ها و وظایف مستقل استفاده می‌کند؛ مهلت‌های محدود، اشتراک منابع و محدودیت‌های تقدم مدل نشده‌اند. چهارم، بحث چندپردازنده‌ای کیفی است نه کمی؛ هیچ زمان‌بندی چندپردازنده‌ای ساخته نشده است. پنجم، مرور ادبیات، هرچند هدفمند، جامع نیست و برخی منابع مرتبط ممکن است حذف شده باشند. ششم، هیچ منبع فارسی‌زبانی به‌طور مستقل راستی‌آزمایی نشد؛ بنابراین دست‌نوشته پوشش منابع دوزبانه را ادعا نمی‌کند. این محدودیت‌ها برای تضمین شفافیت و راهنمایی کارهای آینده بیان شده‌اند.

۳۶. توصیه‌های عملی

بر اساس این تحلیل، توصیه‌های زیر ارائه می‌شود. برای زمان‌بندی دسته‌ای عمومی که در آن کارهای طولانی پرمصرف هستند و اتمام زودهنگام آن‌ها مفید است، LJF می‌تواند در نظر گرفته شود، اما فقط با سازوکارهای پیرسازی یا انتظار محدود برای جلوگیری از گرسنگی کارهای کوتاه. برای سیستم‌های تعاملی، LJF توصیه نمی‌شود. برای سیستم‌های بلادرنگ با وظایف دوره‌ای و مهلت‌های مساوی دوره‌ها، EDF زمانی توصیه می‌شود که بهره‌وری مجموعه‌وظیفه حداکثر ۱ باشد و پیش‌گیری امکان‌پذیر باشد؛ RM زمانی توصیه می‌شود که اولویت‌های ثابت و پیش‌بینی‌پذیری ترجیح داده شوند، مشروط بر آنکه کران بهره‌وری یا تحلیل زمان پاسخ زمان‌بندی‌پذیری را تأیید کند. برای سیستم‌هایی با منابع مشترک، پروتکل‌های ارث‌بری اولویت یا سقف اولویت باید با RM یا EDF برای کاهش وارونگی اولویت استفاده شوند. بن‌بست باید به‌طور مستقل از طریق ترتیب‌دهی منابع، اجتناب یا تشخیص مورد توجه قرار گیرد. در سیستم‌های چندپردازنده‌ای، هیچ سیاست واحدی به‌طور جهانی بهینه نیست؛ انتخاب باید بر اساس بار کاری، هدف (کل زمان اتمام، برآورده‌سازی مهلت، انصاف) و تحلیل زمان‌بندی‌پذیری موجود هدایت شود.

۳۷. جهت‌گیری‌های پژوهشی آینده

کارهای آینده می‌توانند این مطالعه را در چند جهت گسترش دهند. نخست، یک ارزیابی مبتنی بر شبیه‌سازی از EDF، LJF و RM در طیف گسترده‌تری از بارهای کاری (الگوهای ورود متغیر، توزیع‌های زمان انفجار و سطوح بهره‌وری) بینش‌های کمی قابل تعمیم‌تری فراهم می‌کند. دوم، تحلیل LJF با سازوکارهای پیرسازی و مقایسه با SJF با پیرسازی می‌تواند مبادلات بین کاهش گرسنگی و خلوص سیاست را روشن کند. سوم، مطالعه دقیق LJF/LPT چندپردازنده‌ای و رابطه آن با EDF جهانی و RM تقسیم‌شده ارزشمند خواهد بود. چهارم، تعامل بین زمان‌بندی CPU و پروتکل‌های اشتراک منابع (ارث‌بری اولویت، سقف اولویت) شایسته بررسی اختصاصی است. پنجم، یک مرور نظام‌مند ادبیات شامل منابع دانشگاهی فارسی راستی‌آزمایی‌شده بعد دوزبانه را تقویت می‌کند. سرانجام، اندازه‌گیری‌های تجربی روی پلتفرم‌های توکار واقعی به اعتبارسنجی نتیجه‌گیری‌های تحلیلی کمک می‌کند.

۳۸. نتیجه‌گیری

این دست‌نوشته یک تحلیل جامع از الگوریتم زمان‌بندی طولانی‌ترین کار ابتدا (LJF) و یک ارزیابی مقایسه‌ای با زودترین مهلت ابتدا (EDF) و نرخ یکنوا (RM) ارائه کرده است. بخش اول نشان داد که LJF یک سیاست ساده و غیرپیش‌گیرانه است که فرآیند آماده‌ای با بزرگ‌ترین زمان انفجار تخمینی CPU را انتخاب می‌کند. این الگوریتم مستعد گرسنگی کارهای کوتاه است، میانگین زمان انتظار را برای ورودهای هم‌زمان بیشینه می‌کند و یک الگوریتم زمان‌بندی بلادرنگ مبتنی بر مهلت نیست. گونه پیش‌گیرانه آن، LRTF، باید از LJF غیرپیش‌گیرانه متمایز شود. بخش دوم روشن ساخت که EDF، LJF و RM ویژگی‌های متفاوتی را بهینه می‌کنند: LJF به طول کار اولویت می‌دهد، EDF به زودترین مهلت‌ها اولویت می‌دهد و RM اولویت‌های ثابت را بر اساس دوره‌های وظیفه تخصیص می‌دهد. جداول مقایسه‌ای و مثال‌های کارشده نشان دادند که EDF و RM تحت مفروضات مربوطه خود تضمین‌های زمان‌بندی‌پذیری ارائه می‌دهند، در حالی که LJF چنین نمی‌کند. گسترش‌های چندپردازنده‌ای هر سه سیاست غیربدیهی هستند و بن‌بست و وارونگی اولویت به پروتکل‌های مدیریت منابع جداگانه نیاز دارند. سهم این دست‌نوشته یک مقایسه دقیق، خودکفا و تکرارپذیر است که از درهم‌آمیختن مفروضات ناسازگار اجتناب می‌کند. این سند به‌عنوان مرجعی برای پژوهشگران و متخصصانی در نظر گرفته شده است که به دنبال درک ویژگی‌های LJF و رابطه آن با سیاست‌های زمان‌بندی بلادرنگ هستند.

۳۹. منابع

۱. Liu, C. L., and Layland, J. W., “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. ۲۰, no. ۱, pp. ۴۶–۶۱, ۱۹۷۳.

۲. Buttazzo, G. C., “Rate Monotonic vs. EDF: Judgment Day,” *Real-Time Systems*, vol. ۲۹, no. ۱, pp. ۵–۲۶, ۲۰۰۵.

۳. Silberschatz, A., Galvin, P. B., and Gagne, G., *Operating System Concepts*, ۱۰th ed., Wiley, ۲۰۱۸.

۴. Arpaci-Dusseau, R. H., and Arpaci-Dusseau, A. C., *Operating Systems: Three Easy Pieces*, Arpaci-Dusseau Books, ۲۰۱۸.

۵. Stallings, W., *Operating Systems: Internals and Design Principles*, ۹th ed., Pearson, ۲۰۱۷.

۶. Graham, R. L., “Bounds on Multiprocessing Timing Anomalies,” *SIAM Journal on Applied Mathematics*, vol. ۱۷, no. ۲, pp. ۴۱۶–۴۲۹, ۱۹۶۹.

۷. Sha, L., Rajkumar, R., and Lehoczky, J. P., “Priority Inheritance Protocols: An Approach to Real-Time Synchronization,” *IEEE Transactions on Computers*, vol. ۳۹, no. ۹, pp. ۱۱۷۵–۱۱۸۵, ۱۹۹۰.

۸. Coffman, E. G., Elphick, M. J., and Shoshani, A., “System Deadlocks,” *ACM Computing Surveys*, vol. ۳, no. ۲, pp. ۶۷–۷۸, ۱۹۷۱.

۹. Conway, R. W., Maxwell, W. L., and Miller, L. W., *Theory of Scheduling*, Addison-Wesley, ۱۹۶۷.

۱۰. Davis, R. I., and Burns, A., “A Survey of Hard Real-Time Scheduling for Multiprocessor Systems,” *ACM Computing Surveys*, vol. ۴۳, no. ۴, Article ۳۵, ۲۰۱۱.

۱۱. Yoosefan, M., “Operating Systems Course Materials,” Persian university lecture notes, https://yoosofan.github.io/course/os.html (accessed ۲۰۲۵).

۱۲. Kleinberg, J., and Tardos, É., *Algorithm Design*, Pearson, ۲۰۰۵ (for LPT approximation ratio).

۴۰. پیوست الف: محاسبات تکمیلی

راستی‌آزمایی مثال LJF: مجموع انفجار = ۱۰ + ۴ + ۷ + ۲ + ۶ = ۲۹. کل زمان اتمام = ۲۹. زمان‌های اتمام: P۱: ۱۰، P۳: ۱۷، P۵: ۲۳، P۲: ۲۷، P۴: ۲۹. زمان بازگشت: ۱۰، ۱۷، ۲۳، ۲۷، ۲۹. زمان انتظار: ۰، ۱۰، ۱۷، ۲۳، ۲۷. میانگین انتظار = ۷۷/۵ = ۱۵.۴. میانگین بازگشت = ۱۰۶/۵ = ۲۱.۲.

تحلیل زمان پاسخ ۳ → ۳ → ۲ :(\(R_۲ \)). \(R_۱ = ۱ \)**RM:** بنابراین ۳. \(R_۳ \): ۱۰ → ۱۰ → ۹ → ۷ → ۶ → ۳، بنابراین ۱۰. همه ≥ دوره‌های مربوطه.

بهره‌وری \(۰.۸۳۳۳ = ۰.۲۵ + ۰.۳۳۳۳ + ۰.۲۵ \le ۱ \)**EDF:** امکان‌پذیر.

کران بهره‌وری \(U = ۰.۸۳۳۳ > ۰.۷۷۹۷ \approx ۳(۲^{۱/۳} - ۱) \).**RM:** بنابراین آزمون کران غیرقطعی است؛ تحلیل زمان پاسخ زمان‌بندی‌پذیری را تأیید می‌کند.

۴۱. پیوست ب: شبه‌کد تکمیلی و یادداشت‌های تکرارپذیری

```
// LRTF (preemptive LJF) sketch
ALGORITHM LRTF(processes P[۱..n], arrival a[۱..n], burst B[۱..n])
۱. t ← ۰
۲. remaining[i] ← B[i] for all i
۳. ready ← max-heap by remaining time
۴. while unfinished or ready not empty:
۵.     add arrivals with a[i] ≤ t to ready
۶.     if ready empty: t ← next arrival; continue
۷.     i ← extract-max(ready)
۸.     next_event ← min(next arrival time, t + remaining[i])
۹.     run i for (next_event - t)
۱۰.    remaining[i] -= (next_event - t)
۱۱.    t ← next_event
۱۲.    if remaining[i] > ۰: insert i back into ready
۱۳.    else: mark i completed; record completion
۱۴. return schedule
```

یادداشت‌های تکرارپذیری: همه مثال‌ها از شکستن تساوی قطعی استفاده می‌کنند. مثال LJF از شناسه‌های فرآیند به ترتیب صعودی برای تساوی‌ها استفاده می‌کند (هیچ تساوی رخ نمی‌دهد). مثال EDF تساوی‌های مهلت را با زودترین زمان آزادسازی می‌شکند. مثال RM از اولویت‌های ثابت \(\tau_۱ > \tau_۲ > \tau_۳ \) استفاده می‌کند. همه محاسبات را می‌توان با یک صفحه‌گسترده یا یک اسکریپت ساده راستی‌آزمایی کرد. نمودارهای SVG در HTML جاسازی شده‌اند و به منابع خارجی نیاز ندارند. سند برای چاپ A۴ با شکست صفحات CSS طراحی شده است؛ رندر مرورگر ممکن است در صفحه‌بندی کمی متفاوت باشد.

خلاصه مشارکت‌های فنی اصلی: (۱) یک تحلیل مستقل و دقیق از LJF و LRTF با تعاریف صوری، مثال کارشده، شبه‌کد و فلوچارت. (۲) یک تفکیک روشن بین LJF، EDF و RM از نظر هدف بهینه‌سازی و مفروضات. (۳) یک مثال کنترل‌شده مجموعه‌وظیفه EDF/RM با تحلیل بهره‌وری و زمان پاسخ. (۴) جداول مقایسه‌ای که نتیجه‌گیری‌ها را بر اساس بار کاری تعدیل می‌کنند. (۵) یک دست‌نوشته HTML خودکفا و تکرارپذیر با نمودارهای SVG جاسازی‌شده و قالب‌بندی آماده چاپ.

منابع استفاده‌شده: پژوهش بنیادین زمان‌بندی (لیو و لیلند، بوتاتزو)، کتاب‌های درسی استاندارد سیستم‌های عامل (سیلبرشاتز و همکاران، آرپاسی-دوسو و آرپاسی-دوسو، استالینگز)، بررسی‌های زمان‌بندی چندپردازنده‌ای (دیویس و برنز) و ادبیات کلاسیک همگام‌سازی (شا و همکاران، کافمن و همکاران). مطالب درسی فارسی به‌عنوان نقطه شروع ارجاع داده شده اما برای ادعاهای خاص به‌طور مستقل راستی‌آزمایی نشده‌اند؛ هیچ منبع فارسی جعلی گنجانده نشده است.